

UNIVERSIDADE PAULISTA – UNIP
GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO
CC8P33

BRUNO DA SILVA PIMENTEL – N844132
GUILHERME VIEIRA ABBENANTE GOMES – N8959E0
HITALO CHAVES DOS SANTOS – N841419
JOÃO VICTOR CRISCI – F3445H0

DETECÇÃO AUTOMATIZADA DE DESCARTE IRREGULAR DE LIXO

SÃO PAULO

2025

UNIVERSIDADE PAULISTA – UNIP
GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO
CC8P33

BRUNO DA SILVA PIMENTEL - N844132
GUILHERME VIEIRA ABBENANTE GOMES - N8959E0
HITALO CHAVES DOS SANTOS - N841419
JOÃO VICTOR CRISCI – F3445H0

DETECÇÃO AUTOMATIZADA DE DESCARTE IRREGULAR DE LIXO

Trabalho de Atividade Prática Supervisionada
para obtenção de nota final do curso de
Ciência da Computação apresentado à
Universidade Paulista – UNIP.

Orientador: Prof. Dr. Luiz Carlos Magrini

SÃO PAULO - SP

2025

RESUMO

O descarte irregular de lixo é um problema crescente nos centros urbanos, impactando diretamente a saúde pública, o meio ambiente e a qualidade de vida. Resíduos abandonados em vias públicas, terrenos baldios e margens de corpos hídricos facilitam a proliferação de vetores de doenças, causam poluição visual e ambiental, e oneram os cofres públicos com gastos adicionais de limpeza. Embora a Política Nacional de Resíduos Sólidos e legislações municipais estabeleçam regras claras para coibir essas infrações, a fiscalização tradicional enfrenta desafios como a limitação de recursos humanos, a cobertura ineficiente de grandes áreas e a dificuldade de monitoramento contínuo, tornando o problema persistente e de difícil controle. Diante desse cenário, este trabalho propõe uma solução inovadora baseada em Visão Computacional e Inteligência Artificial para a detecção automatizada de focos de descarte irregular de lixo. O sistema desenvolvido utiliza técnicas avançadas de processamento de imagens e aprendizado de máquina para identificar a presença de resíduos em locais inadequados, como logradouros públicos, áreas de preservação permanente e pontos não autorizados. Além de automatizar o processo de fiscalização, a solução visa aumentar a eficiência na identificação de infrações ambientais, reduzir a recorrência de descartes irregulares e auxiliar na formulação de políticas públicas voltadas para a gestão integrada de resíduos sólidos. Para validar a abordagem proposta, serão conduzidos testes em diferentes cenários urbanos, analisando a precisão e a confiabilidade da detecção automatizada em comparação com os métodos tradicionais de fiscalização.

Palavras-Chave: Descarte irregular de lixo; Resíduos Sólidos; Visão Computacional; Inteligência Artificial; Detecção Automatizada.

ABSTRACT

Irregular waste disposal is a growing problem in urban centers, directly impacting public health, the environment, and the quality of life. Waste dumped in public spaces, vacant lots, and along water bodies facilitates the proliferation of disease vectors, causes visual and environmental pollution, and burdens public coffers with additional cleaning costs. Although the Brazilian National Solid Waste Policy and municipal legislation establish clear rules to curb these infractions, traditional enforcement faces challenges such as limited human resources, inefficient coverage of large areas, and the difficulty of continuous monitoring, making the problem persistent and hard to control. Given this scenario, this work proposes an innovative solution based on Computer Vision and Artificial Intelligence for the automated detection of irregular waste disposal sites. The developed system uses advanced image processing and machine learning techniques to identify the presence of waste in unsuitable locations, such as public grounds, permanent preservation areas, and unauthorized spots. In addition to automating the enforcement process, the solution aims to increase the efficiency in identifying environmental offenses, reduce the recurrence of illegal dumping, and assist in the formulation of public policies aimed at integrated solid waste management. To validate the proposed approach, tests will be conducted in different urban scenarios, analyzing the accuracy and reliability of automated detection compared to traditional inspection methods.

Keywords: Infraction; Irregular waste disposal; Computer Vision; Artificial Intelligence; Automated Detection.

SUMÁRIO

1 INTRODUÇÃO	7
1.1 Contextualização	7
1.2 Problema	7
1.3 Objetivo	8
1.3.1 Objetivo Geral	8
1.3.2 Objetivos Específicos	8
2 REFERENCIAL TEÓRICO.....	10
2.1 Inteligência Artificial.....	10
2.1.1 Redes Neurais Artificiais.....	11
2.2 Visão computacional	14
2.2.1 Captura de imagem.....	14
2.2.2 <i>You Only Live Once</i> (Yolo)	14
2.2.3 Detecção de resíduos sólidos (lixo).....	15
2.3 Sistemas distribuídos.....	16
2.4 Regulamentação Brasileira.....	17
2.4.1 Infrações.....	17
3 PLANO DE DESENVOLVIMENTO	18
3.1 Documento de Visão	18
3.1.1 Nome do projeto	18
3.1.2 Objetivo Geral	18
3.1.3 Justificativa	19
3.1.4 Objetivos Específicos	19
3.1.5 Escopo do Projeto	19
3.1.6 Stakeholders Acadêmicos	20
3.1.7 Resultados Esperados.....	20
3.1.8 Arquitetura do sistema e ferramentas utilizadas.....	20
3.1.9 Tecnologias Sugeridas.....	20
3.1.10 Restrições e Considerações	21
3.2 Documento de requisitos (funcionais e não funcionais).....	21
3.2.1 Requisitos Funcionais (Casos de uso).....	21
3.2.1.1 [UC01] Cadastrar usuário	21
3.2.1.2 [UC02] Fazer login do usuário	21
3.2.1.3 [UC03] Enviar Arquivo	22

3.2.1.4 [UC04] Consultar Ocorrências de descarte irregular de lixo.....	22
3.2.1.5 [UC05] Detectar elementos desejados	22
3.2.1.6 [UC06] Extrair dados da ocorrência	22
3.2.1.7 [UC07] Armazenar os dados.....	22
3.2.1.8 [UC08] Notificar o usuário.....	22
3.2.2 Requisitos Não Funcionais	22
3.2.2.1 Estabilidade Funcional	22
3.2.2.2 Eficiência de Desempenho	22
3.2.2.3 Compatibilidade.....	23
3.2.2.4 Usabilidade.....	23
3.2.2.5 Confiabilidade.....	23
3.2.2.6. Segurança	24
3.2.2.7 Manutenibilidade	24
3.2.2.8 Portabilidade.....	24
3.3 Arquitetura de Solução	24
3.4 Projeto lógico do banco de dados	25
3.5 Projeto físico do banco de dados	27
3.6 Modelagem do Software	28
3.6.1 Diagrama de Caso de Uso.....	29
3.6.2 Documentação de caso de uso.....	31
3.6.3 Diagrama de Classe	39
3.6.4 Diagramas de Sequência	40
3.6.4.1 DS Cadastro.....	40
3.6.4.2 DS Login.....	42
3.6.4.3 DS Enviar Arquivo	44
3.6.4.4 DS Consultar Ocorrências	45
3.6.4.5 DS Fluxo completo de detecção.....	47
3.6.4.6 DS Notificação	49
4 PROJETO DO PROGRAMA.....	50
4.1 Back-End (Servidor).....	50
4.1.1 Fluxo de processamento da imagem.....	50
4.2 Front-End (Cliente).....	52
5 CÓDIGO FONTE.....	53
6 REFERÊNCIA BIBLIOGRAFICA.....	54
7 FICHAS DE ATIVIDADE PRÁTICA SUPERVISIONADA (APS)	57

1 INTRODUÇÃO

O presente trabalho visa criar um sistema automático para monitorar o descarte irregular de lixo em ambientes urbanos, empregando visão computacional e Inteligência Artificial (IA). O objetivo é aumentar a eficácia no controle dessas transgressões ambientais, diminuir a necessidade de fiscalização humana e auxiliar numa gestão de limpeza urbana mais inteligente e ecologicamente correta.

A solução planejada utiliza tecnologias como YOLOv11 (You Only Look Once) para identificar resíduos ou aglomerados de lixo instantaneamente. O sistema examina fotos e vídeos de câmeras de vigilância urbanas, detectando a presença de lixo em locais proibidos (terrenos baldios, vias públicas, margens de corpos hídricos, dentre outras), estimando o volume do descarte e gerando relatórios estatísticos para análise e ação das autoridades competentes.

1.1 Contextualização

À medida que as cidades se desenvolvem e o consumo aumenta, a geração de resíduos sólidos urbanos cresce significativamente, criando um desafio para a gestão pública. Um dos problemas mais críticos decorrentes disso é o aumento dos casos de descarte irregular de lixo, cada vez mais comuns em áreas urbanas com pouca fiscalização, gerando impactos ambientais, sanitários e estéticos.

Nesse contexto, o uso de novas tecnologias, como a visão computacional e a inteligência artificial, tem surgido como um forte apoio para apresentar formas automáticas de identificar o descarte irregular de lixo.

1.2 Problema

Garantir a salubridade e a qualidade ambiental nas áreas urbanas é vital para o bem-estar da população, impactando diretamente a saúde pública e a sustentabilidade das cidades. A adoção de práticas de gestão de resíduos eficientes desempenha papel crucial na melhoria da qualidade de vida, ao minimizar a poluição, reduzir a proliferação de vetores de doenças e preservar os ecossistemas urbanos.

No contexto contemporâneo, a supervisão de infrações ligadas ao descarte irregular de resíduos ainda se baseia majoritariamente em denúncias e no

monitoramento manual por agentes fiscais. Isso demanda a presença física em locais específicos e acarreta diversas restrições, tais como a vulnerabilidade a erros humanos, a dificuldade de cobertura de grandes áreas e a resposta tardia às irregularidades. Tais limitações acabam permeando uma sensação de impunidade, que, por sua vez, influencia o aumento de novos focos de descarte inadequado.

Considerando a situação atual, fica clara a necessidade de um método aprimorado, que consiga detectar, de maneira precisa e constante, as violações ligadas ao descarte irregular de lixo. Assim, seria viável acelerar a resposta dos órgãos de fiscalização ambiental e de limpeza urbana e, conseqüentemente, desestimular a população a cometer esse tipo de infração.

1.3 Objetivo

1.3.1 Objetivo Geral

Desenvolver um sistema automatizado baseado em inteligência artificial e visão computacional, para detecção e fiscalização de descarte irregular de lixo em áreas urbanas. Otimizando a gestão da limpeza urbana e contribuindo para a preservação ambiental ao reduzir a ocorrência de infrações relacionadas ao descarte inadequado de resíduos por meio de uma atuação mais precisa e dinâmica.

1.3.2 Objetivos Específicos

- Implementar através de algoritmos de visão computacional (como YOLOv11) a habilidade de analisar, identificar e capturar automaticamente, através de foto, o lixo descartado irregularmente;
- Implementar a identificação de características gerais como local, data e hora da ocorrência;
- Armazenar os dados registrados em um banco de dados, para futura análise e consulta das ocorrências registradas;
- Produzir uma interface, conectado ao sistema, que permita ao usuário:
 - A validação direta de ocorrências de descarte através do upload de foto;
 - Consulta das ocorrências cometidas através da pesquisa local;

- Visualização de mapa que contém a análise de características gerais de ocorrências, como local, data, hora etc.

2 REFERENCIAL TEÓRICO

2.1 Inteligência Artificial

A Inteligência Artificial é um campo da tecnologia, que busca fazer com que os computadores ajam ou pensem de maneira inteligente (GOMES, 2010). Para isso, o principal objetivo da IA é gerar uma máquina que seja capaz de imitar o comportamento humano de forma inteligente, por meio de linguagens naturais, reconhecimento de padrões e aprender com base em experiências prévias de tomadas de decisões baseadas em dados (TAULLI, 2020).

Segundo TAULLI (2020), Inteligência Artificial pode ser classificada em dois tipos:

- IA Fraca: essa se limita a identificação de padrões (*pattern matching*), além de focar em tarefas específicas, como a assistente virtual Alexa, da Amazon, por exemplo.
- IA Forte: é a fortemente conhecida por filmes de ficção científica, no qual ela realmente possui compreensão da realidade inserida, podendo demonstrar criatividade e emoções.

Com base nisso, para TAULLI (2020), a Inteligência Artificial está localizada inicialmente na IA Fraca, e o objetivo dependendo do caso é alcançar o nível de IA Forte, o que pode levar anos e até décadas, apesar de que há pesquisadores que não acreditam nesta possibilidade.

Como comentado por GOMES (2010), Alan Turing, conhecido como o pai da informática, foi o pioneiro a ter uma visão abrangente da IA, como apresentado em seu artigo "*Computing Machinery and Intelligency*", em 1950, no qual propôs o Teste de Turing. Esse teste consiste em avaliar a capacidade de um computador em exibir comportamento inteligente indistinguível ao de um ser humano, de modo que um interrogador, ao realizar perguntas por escrito, não seja capaz de determinar se as respostas foram enviadas por um humano ou por uma máquina.

Dentro da área de Inteligência Artificial, há o subcampo *Machine Learning* (ML), que foca em desenvolver algoritmos que são capazes de aprender padrões por meio de dados, sem que sejam explicitamente programados (MITCHELL, 1997). Segundo

MITCHELL (1997, p. 2), "um programa de computador é dito aprender com a experiência E em relação a alguma classe de tarefas T e medida de desempenho P , se seu desempenho em T , medido por P , melhora com a experiência E ". Para ALPAYDIN (2020), o ML pode ser definido como uma área da ciência que busca capacitar os computadores a aprenderem com base em dados, aprimorando seu desempenho em atividades específicas por meio da experiência.

Dentro de Machine Learning, destacam-se duas categorias principais. O aprendizado supervisionado envolve modelos treinados com dados já rotulados, como classificação de imagens, enquanto o aprendizado não supervisionado identifica padrões em dados que não estão rotulados, como *clustering* (BISHOP, 2006). Segundo HASTIE et al. (2009), pode-se comparar um estudante que aprende com seu professor, onde os rótulos agem como feedback correto, como um exemplo análogo de aprendizado supervisionado.

Por fim, observa-se que a aplicação das técnicas e conceitos abordados é essencial, uma vez que o sistema desenvolvido lida com imagens de ambientes urbanos que demandam reconhecimento de padrões. Dessa forma, torna-se possível a identificação automatizada dos descartes irregulares de lixo cometidos, a partir de modelos treinados, eliminando a necessidade de intervenção humana no processo de reconhecimento.

2.1.1 Redes Neurais Artificiais

A trajetória das redes neurais artificiais tem início com o modelo proposto por McCulloch e Pitts, em 1943, que buscava representar o funcionamento dos neurônios biológicos por meio de circuitos lógicos. Esse trabalho pioneiro lançou as bases teóricas para os avanços que viriam nas décadas seguintes, como o Perceptron desenvolvido por Rosenblatt em 1958, e, mais recentemente, as redes neurais profundas. De acordo com GOODFELLOW, BENGIO e COURVILLE (2016), um dos marcos mais importantes dessa evolução foi a popularização do algoritmo de retropropagação, na década de 1980, que possibilitou o treinamento eficaz de redes com múltiplas camadas. Esse avanço foi essencial para que as redes neurais passassem a ser aplicadas com sucesso em áreas como a visão computacional e o reconhecimento de padrões complexos, temas que também são abordados na seção 2.2.

As redes neurais artificiais (RNAs), são modelos de computadores que visam simular o sistema nervoso humano e biológico. HAYKIN (2001) define as redes neurais como processadores distribuídos e altamente paralelizados, constituídos por unidades de processamento básicas, que possuem uma capacidade inerente de armazenar conhecimento adquirido experimentalmente e disponibilizá-lo quando necessário. Tal característica encaixa perfeitamente para o sistema proposto neste trabalho, onde é preciso haver aprendizado adaptativo para que padrões de imagens de resíduos sólidos considerados complexos, possam ser identificados.

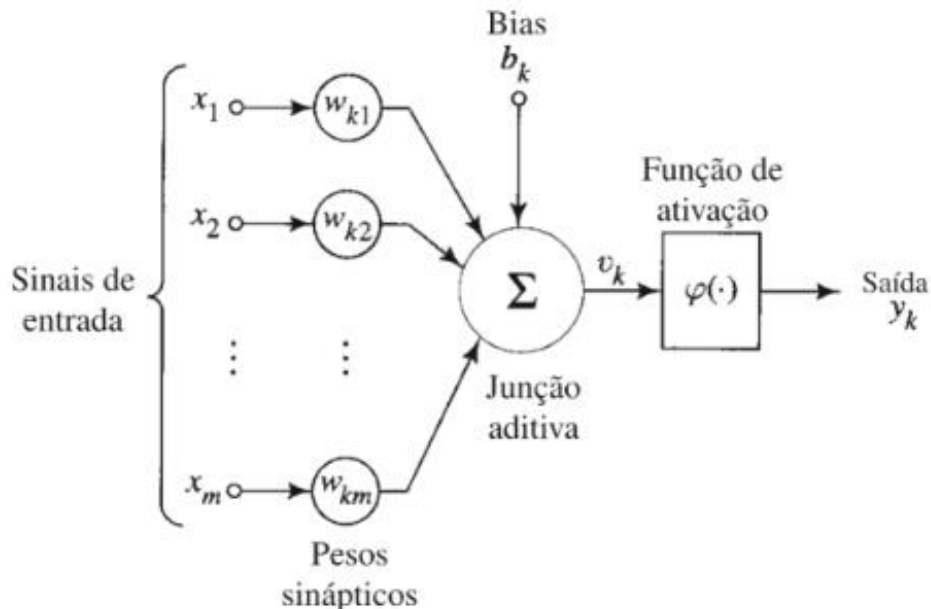
Uma rede neural é formada por três camadas essenciais: a de entrada (*input layer*), as intermediárias (*hidden layers*) e a de saída (*output layer*). A primeira recebe os dados iniciais, como os pixels de uma imagem, enquanto as intermediárias processam essas informações por meio de operações matemáticas para identificar padrões importantes. Por fim, a camada de saída gera o resultado, como a identificação de um descarte irregular de lixo. (RUSSELL; NORVIG, 2016).

Segundo HAYKIN (2001, p. 36), o neurônio artificial representa a unidade básica de processamento de informação em uma rede neural, sendo estruturalmente composto por três elementos principais:

1. Conjunto de sinapses (conexão de entrada), cada uma com um peso individual.
2. Um somador, que combina os sinais de entrada.
3. Uma função de ativação, para limitar a amplitude de saída

A rede neural mais simples, chamada de Perceptron, contém apenas um neurônio. (Figura 1)

Figura 1 - Modelo de Neurônio Artificial



Fonte: HAYKIN (2001)

Para que uma rede seja desenvolvida com eficiência, é essencial que ela passe pela etapa de Machine Learning (ML), uma vez que esse processo de treinamento é fundamental para que o sistema compreenda e atue adequadamente em uma área específica (BIANCHI, 2004). Essa abordagem está alinhada com a visão de ALPAYDIN (2020), que define o ML como um subcampo da Inteligência Artificial (IA) voltado para capacitar computadores a aprender com dados, melhorando seu desempenho em tarefas específicas por meio da experiência.

Neste projeto, as redes neurais serão aplicadas em dois estágios principais: detecção de objetos, por meio da arquitetura YOLO (*You Only Look Once*), responsável pela identificação de resíduos sólidos e outros elementos urbanos em imagens capturadas por câmeras, de acordo com o tópico 2.3.2, o YOLO é um modelo que combina precisão e velocidade de detecção em tempo real (REDMON et al., 2016).

A Inteligência Artificial, particularmente com o uso de técnicas como Machine Learning e Redes Neurais Artificiais, tem se destacado como uma ferramenta eficiente na automação de tarefas complexas. Essas tecnologias são capazes, por exemplo,

de identificar padrões em imagens e identificar descartes irregulares de lixo com precisão, otimizando processos que demandariam grande esforço humano. Segundo RUSSELL; NORVIG (2016), o aprendizado de máquina possibilita que os sistemas aprimorem seu desempenho de forma contínua por meio da experiência, aumentando assim sua eficiência em aplicações específicas. Logo, a aplicação dessas tecnologias no reconhecimento automatizado de infrações não só otimiza processos, mas também reforça a importância da IA como um campo em constante evolução, capaz de transformar a maneira como interagimos com sistemas computacionais.

2.2 Visão computacional

A visão computacional, é uma maneira de fazer com que computadores extraiam elementos e/ou características de vídeos e imagens, através de redes neurais e *Machine Learning* (Aprendizado de máquina) (IMB, 2019).

2.2.1 Captura de imagem

A captura de imagens pode ser feita através de dispositivos de entrada, como uma câmera que grava um vídeo ou através de um upload direto da imagem digital. É importante lembrar que para funcionar corretamente com uma ferramenta de visão computacional, principalmente se precisar implementar de maneira em tempo real, o vídeo acaba sendo transferido como um compilado de frames para ser processado, então muitas vezes alguns dos frames são perdidos para a visualização do resultado final assim, tendo um resultado com menor frame rate (BERTOLETI, 2017). Porém para esse tipo de aplicação não é um problema, pois não será necessário ter resultados suaves, mas sim precisos.

2.2.2 You Only Live Once (Yolo)

O modelo de detecção de objetos e segmentação de imagens em tempo real, YOLO, pela empresa Ultralytics, é conhecido pela sua rapidez e precisão. É capaz de identificar e localizar objetos em imagens e vídeos, é possível aplicá-lo na robótica, como em veículos autônomos e em sistemas de vigilância (KUNDU, 2023) (Imagem 1)

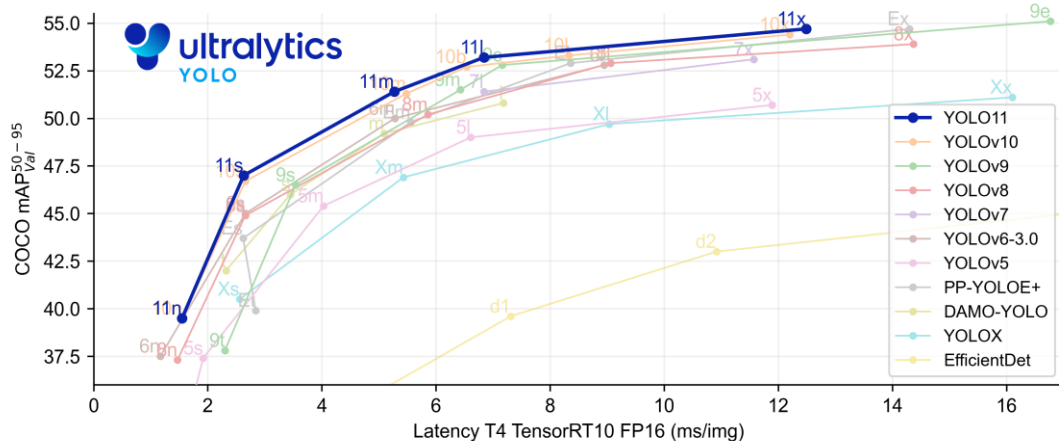
Imagem 1 - Gerenciamento de Estacionamento em cidades Inteligentes com YOLOv8



Fonte: VINA, 2025

A versão mais recente é o YOLOv11, oferecendo um desempenho bem maior que suas versões anteriores, com alta acurácia e baixa latência. É considerado um modelo SOTA (*State of the Art*), ou seja, de alta performance para realizar tarefas específicas de Inteligência artificial (SRIVASTAVA, 2025). Oferece também ajuda nas tarefas de estimativa de pose, rastreamento e classificação (ULTRALYTICS, 2025). (Gráfico 2)

Gráfico 1 - Comparação de Performance YOLOv11



Fonte: Ultralytics, 2024

2.2.3 Detecção de resíduos sólidos (lixo)

Também é possível treinar o YOLO, alimentando-o com um *dataset* (conjunto de dados, nesse caso imagens) customizado e ajustando os parâmetros, para assim poder identificar e localizar elementos específicos. É necessário que o conjunto de imagens esteja rotulado/classificado com as classes específicas, ou seja, quais são e

onde estão localizados os elementos desejados. Para isso já existem fontes com *datasets* prontos, como o Roboflow, Ultralytics Hub e Google Open Images v7. Mas mesmo que precise rotular um dataset do zero, é possível utilizar ferramentas como o Label Studio para facilitar a classificação (ULTRALYTICS, 2025). (Imagem 2)

Imagem 2 - Exemplos de datasets para detecção de resíduos sólidos (lixo)



Fonte: Roboflow, 2025

2.3 Sistemas distribuídos

Os Sistemas Distribuídos constituem um campo da computação voltado ao estudo e desenvolvimento de ambientes onde diversos computadores independentes trabalham de forma coordenada para atingir um objetivo comum. Nesses sistemas, os recursos computacionais e dados encontram-se fisicamente distribuídos, mas são interligados por redes de comunicação que permitem o compartilhamento de informações e a execução colaborativa de tarefas. O propósito central é proporcionar maior escalabilidade, confiabilidade e desempenho do que seria possível em um sistema isolado.

A organização de um sistema distribuído permite que seus componentes funcionem como uma unidade lógica única, ainda que estejam geograficamente dispersos. Para isso, são utilizadas técnicas de sincronização, comunicação e tolerância a falhas que possibilitam ao usuário interagir com o ambiente como se fosse um sistema único. Essa abordagem é aplicada em diversos cenários, como plataformas de computação em nuvem, redes de entrega de conteúdo e aplicações empresariais que exigem alta disponibilidade.

Além disso, sistemas distribuídos são fundamentais para suportar aplicações modernas que demandam grande capacidade de processamento e acesso rápido a dados. Por meio de mecanismos de balanceamento de carga, replicação de serviços e escalabilidade dinâmica, essas infraestruturas conseguem atender a milhões de

usuários simultaneamente, mantendo desempenho estável e maior resiliência frente a falhas. Essa arquitetura tem se tornado cada vez mais essencial para viabilizar desde serviços de streaming até transações financeiras em larga escala.

2.4 Regulamentação Brasileira

A Política Nacional de Resíduos Sólidos (PNRS), instituída pela Lei nº 12.305/2010, é o marco regulatório fundamental para a gestão integrada e o gerenciamento ambientalmente adequado de resíduos sólidos no Brasil. No contexto deste trabalho, a PNRS assume especial relevância, pois estabelece os princípios, objetivos e instrumentos para o correto descarte de resíduos, definindo as responsabilidades dos geradores e do poder público, e servindo como base legal para o desenvolvimento deste sistema automatizado de detecção de descarte irregular de lixo.

2.4.1 Infrações

No âmbito da Política Nacional de Resíduos Sólidos (PNRS) e tipificada pela Lei de Crimes Ambientais (Lei nº 9.605/1998), o descarte irregular de resíduos é caracterizado como uma conduta lesiva ao meio ambiente. A PNRS veda expressamente a disposição final inadequada de resíduos sólidos em locais como logradouros públicos, terrenos baldios, corpos hídricos e áreas protegidas (BRASIL, 2010, art. 47). Essas condutas, consideradas crimes ambientais, acarretam penalidades específicas, como multas, obrigação de reparar os danos causados e, em casos graves, até mesmo restrição de direitos e reclusão dos responsáveis.

3 PLANO DE DESENVOLVIMENTO

O Capítulo 3 apresenta a modelagem técnica da solução proposta para detecção automatizada de descarte irregular de lixo. São descritas as principais estruturas que compõem o sistema, iniciando-se pelo documento de visão, que descreve informações gerais sobre o projeto do sistema, agindo como um complemento da introdução.

Na sequência é apresentado o documento de requisitos, que se divide em dois: Requisitos Funcionais, que são as funções exatas que o sistema busca efetuar (também chamados de casos de uso), e os Requisitos Não Funcionais, que definem como as funcionalidades serão alcançadas, através de critérios como eficiência, usabilidade, segurança etc.

Logo em seguida vem a arquitetura da solução, a qual define a organização e a interação entre os componentes responsáveis pelo processamento de imagens, detecção de resíduos sólidos, interface com o usuário e persistência de dados.

Posteriormente, é detalhado o projeto lógico e físico do banco de dados, contemplando a estrutura relacional utilizada para armazenamento das informações processadas como os metadados da imagem e resultados das detecções realizadas pelo sistema.

Este capítulo visa estabelecer uma visão clara, estruturada e fundamentada da solução proposta, o que permite compreender da melhor forma como o projeto de software está planejado e organizado.

3.1 Documento de Visão

3.1.1 Nome do projeto

EcoDetect

3.1.2 Objetivo Geral

Criar um sistema automático para monitorar o descarte irregular de lixo em ambientes urbanos, empregando visão computacional e Inteligência Artificial (IA). O objetivo é aumentar a eficácia no controle dessas transgressões ambientais, diminuir

a necessidade de fiscalização humana e auxiliar numa gestão de limpeza urbana mais inteligente e ecologicamente correta.

3.1.3 Justificativa

O crescimento populacional e o consumo desenfreado têm agravado severamente o problema do descarte irregular de resíduos sólidos nas cidades. Dados da Associação Brasileira de Empresas de Limpeza Pública e Resíduos Especiais (Abrelpe) indicam que milhões de toneladas de resíduos são descartadas inadequadamente anualmente no país, reflexo de uma fiscalização limitada e ineficiente. A atuação manual dos agentes de fiscalização é cara, restrita e sujeita a falhas, o que favorece a impunidade e a recorrência dessas infrações ambientais, com sérios impactos na saúde pública e na qualidade de vida.

Nesse contexto, o uso de tecnologias como visão computacional e inteligência artificial oferece uma solução moderna e eficaz, permitindo a detecção automática e contínua de focos de descarte irregular. Além de otimizar a fiscalização, essas tecnologias contribuem para a preservação do meio ambiente, a redução de gastos públicos com limpeza e a promoção de cidades mais sustentáveis.

3.1.4 Objetivos Específicos

Estudar e comparar técnicas de visão computacional aplicadas à detecção de descarte irregular de resíduos sólidos.

Implementar um protótipo funcional capaz de identificar automaticamente situações de descarte ilegal.

Utilizar métricas adequadas para avaliar o desempenho do sistema com precisão.

Desenvolver uma interface simples para visualização dos resultados e interação com os usuários.

Documentar todas as etapas do desenvolvimento, incluindo decisões técnicas, testes e resultados obtidos.

3.1.5 Escopo do Projeto

Sistema web funcional com banco de dados.

Cadastro de usuários.

Perfil do usuário, histórico de ocorrências e preferências.

Busca e visualização de ocorrências.

Banco de dados para treinamento da IA.

Sistema de validação e reconhecimento de lixo.

Sistema de extração de metadados.

3.1.6 Stakeholders Acadêmicos

Equipe de desenvolvimento: alunos participantes do projeto.

Professor/orientador: supervisiona o progresso técnico e metodológico.

Usuários *testers*: colegas ou voluntários que interagem com o sistema para fins de validação.

Instituição de Ensino: apoia o projeto com infraestrutura e recursos.

3.1.7 Resultados Esperados

Protótipo funcional de um sistema automatizado para detecção de descarte irregular de lixo por meio de visão computacional.

Relatório técnico contendo:

Revisão bibliográfica sobre descarte irregular e tecnologias aplicadas.

3.1.8 Arquitetura do sistema e ferramentas utilizadas.

Modelos de visão computacional e algoritmos aplicados, com justificativa de escolha.

Avaliação do desempenho com métricas quantitativas (ex: acurácia, precisão, recall).

Potencial de aplicação prática em cidades inteligentes e uso como base para pesquisas futuras.

3.1.9 Tecnologias Sugeridas

Backend: Python

Frontend: HTML, CSS e JavaScript

Banco de Dados: MYSQL

Bibliotecas e ferramentas: YOLO

3.1.10 Restrições e Considerações

O desenvolvimento do sistema está sujeito a algumas restrições técnicas e operacionais. A principal limitação está na disponibilidade e qualidade das imagens utilizadas, que podem variar conforme o ambiente, iluminação, ângulo da câmera e presença de obstáculos visuais. Essas variáveis podem impactar diretamente a acurácia do modelo de visão computacional.

Além disso, o treinamento de algoritmos de detecção exige uma base de dados robusta e representativa, o que demanda tempo e recursos para coleta e rotulagem adequada. Outra consideração relevante envolve questões legais e éticas, como o uso de imagens captadas em espaços públicos, respeitando a privacidade dos indivíduos e a legislação vigente.

Do ponto de vista técnico, o desempenho do sistema poderá variar conforme o poder computacional disponível, principalmente em aplicações em tempo real. Assim, será necessário buscar um equilíbrio entre eficiência, custo e precisão. Por fim, vale considerar que o sistema proposto não substituirá totalmente a atuação humana, mas atuará como ferramenta de apoio à fiscalização, ampliando sua efetividade e alcance.

3.2 Documento de requisitos (funcionais e não funcionais)

3.2.1 Requisitos Funcionais (Casos de uso)

3.2.1.1 [UC01] Cadastrar usuário

O sistema deve permitir ao usuário realizar seu cadastro por meio do preenchimento de suas credenciais em uma etapa inicial.

3.2.1.2 [UC02] Fazer login do usuário

O sistema deve permitir ao usuário acessar sua conta por meio do preenchimento de suas credenciais de login.

3.2.1.3 [UC03] Enviar Arquivo

O sistema deve permitir ao usuário efetuar o envio de um arquivo de imagem, a fim de validar uma ocorrência.

3.2.1.4 [UC04] Consultar Ocorrências de descarte irregular de lixo

O sistema deve permitir ao usuário consultar as ocorrências de descarte irregular de lixo registradas no banco de dados.

3.2.1.5 [UC05] Detectar elementos desejados

O sistema deve mandar uma requisição para uma API externa que possa detectar passivamente os elementos presentes em imagens utilizadas como entrada, se presentes na mesma.

3.2.1.6 [UC06] Extrair dados da ocorrência

O sistema deve extrair as características relevantes do momento da ocorrência, utilizando-se da requisição para uma API que extraia o horário, data e hora do ocorrido através dos metadados.

3.2.1.7 [UC07] Armazenar os dados

O sistema deve armazenar no banco de dados todas as informações extraídas pela Inteligência Artificial, mantendo a integridade e rastreabilidade dos registros.

3.2.1.8 [UC08] Notificar o usuário

O sistema deve exibir uma notificação (pop-up) ao usuário informando que uma detecção de descarte irregular de lixo foi de fato identificada, ou não.

3.2.2 Requisitos Não Funcionais

3.2.2.1 Estabilidade Funcional

O sistema busca ser estável a partir do momento em que o software identifica um frame de uma entrada visual como válido, pois assim será possível a execução do algoritmo para obter algum tipo de resposta, mesmo se a resposta voltar como nula (nenhum lixo detectado).

3.2.2.2 Eficiência de Desempenho

O sistema desenvolvido deverá processar os dados e executar o algoritmo de uma só vez, ao buscar percorrer o fluxo completo do envio da imagem até o resultado esperado. Além de contar com uma *API Rest Full*, no qual suas requisições e respostas terão um tempo estimado de 15 segundos.

3.2.2.3 Compatibilidade

A presente aplicação apresenta compatibilidade pois integra de forma eficaz o *front-end* em HTML, CSS e JavaScript com o *back-end* em Python, permitindo que ambos se comuniquem por meio de APIs padronizadas e formatadas em JSON, dessa forma garantindo que as informações transmitidas entre a interface do usuário e a IA de análise e validação de imagens, sejam interpretadas corretamente por cada componente, assim tornando o funcionamento do sistema coeso e sem conflitos ou necessidade de adaptações complexas, assegurando um fluxo contínuo de dados e uma operação estável.

3.2.2.4 Usabilidade

O sistema desenvolvido prioriza uma experiência intuitiva e eficiente, garantindo que o usuário consiga realizar todas as etapas, desde o envio da imagem até a visualização da ocorrência identificada, de forma fluida e sem complicações. A interface foi projetada para minimizar cliques desnecessários, além de conter menus simplificados, linguagem direta e orientações contextualizadas, que complementam a usabilidade, tornando a fiscalização mais rápida e reduzindo a curva de aprendizado.

3.2.2.5 Confiabilidade

O sistema deverá apresentar acurácia mínima de 85% na detecção de resíduos ou aglomerados de lixo, considerando que o modelo de visão computacional deverá ser treinado com base em um conjunto de dados diversificado, incluindo diferentes tipos de resíduos sólidos, ângulos, perspectivas e posicionamento dos mesmos, para garantir robustez.

Além de uma disponibilidade total de 24 horas por dia de operação, em caso de falha o sistema deverá ser capaz de tratar o erro e continuar suas operações sem comprometer a disponibilidade e a segurança. O sistema deverá ser capaz de detectar

e descartar falsos positivos com base em uma política de verificação dupla ou análise por IA adicional, quando necessário.

3.2.2.6. Segurança

O sistema fará uso de criptografia TLS/SSL de ponta a ponta, assegurando a proteção dos dados transmitidos entre o cliente e o servidor. Além disso, será implementado um mecanismo de autenticação de usuários, garantindo que cada indivíduo tenha acesso exclusivamente ao seu próprio perfil.

A gestão de permissões será aplicada de forma individualizada, permitindo que cada usuário realize apenas as operações de inserção, exclusão e atualização relativas aos seus próprios dados. Da mesma forma, o acesso às informações será estritamente restrito ao respectivo perfil, sendo vedado o acesso a contas de terceiros. Dessa forma, cada perfil permanece único, privado e intransferível.

3.2.2.7 Manutenibilidade

O sistema busca precisar de poucas manutenções pois, para efetuar a testabilidade será necessário treinar o modelo de detecção com quantidades consideráveis de imagens (*datasets*), sendo elas de diferentes condições impostas. Além do algoritmo em si que, uma vez que estiver entregando algum resultado, significa que o fluxo inteiro foi percorrido com sucesso.

3.2.2.8 Portabilidade

O sistema desenvolvido será compatível com qualquer dispositivo eletrônico no qual tenha suporte a um navegador web, seja ele PC ou mobile.

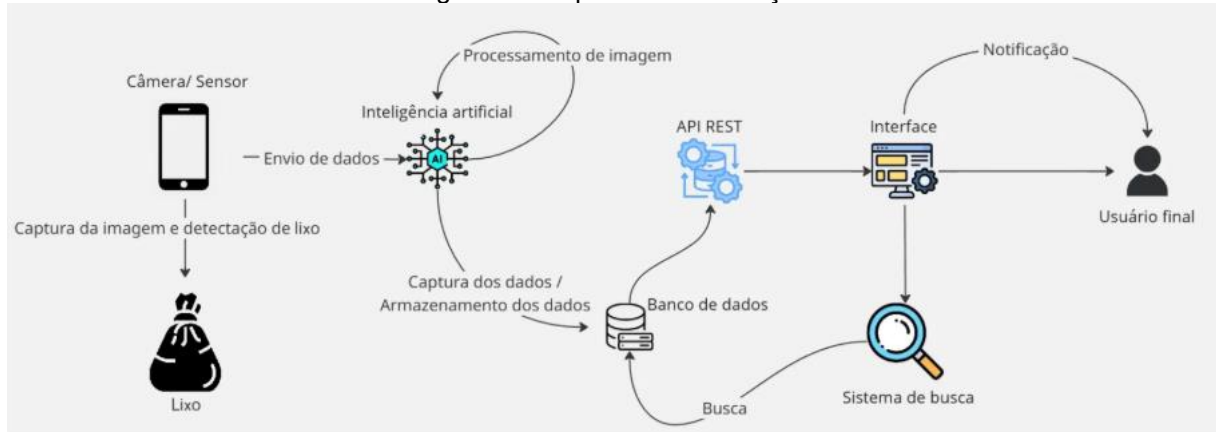
3.3 Arquitetura de Solução

A arquitetura de solução do sistema (figura 2) é composta por diversos módulos integrados que viabilizam a detecção automatizada de descarte irregular de lixo. O processo tem início com a captura de imagens por câmeras ou sensores, em seguida essas imagens são enviadas para um módulo de inteligência artificial, que realiza o processamento e identificação de lixo descartado irregularmente.

Os dados extraídos são capturados e armazenados em um banco de dados, sendo disponibilizados por meio de uma API REST para a interface do sistema que

consome esses dados e permite que os usuários realizem buscas e visualizações de forma eficiente. Por fim, o sistema notifica o usuário final sempre que uma ocorrência é identificada.

Figura 2 – Arquitetura de Solução

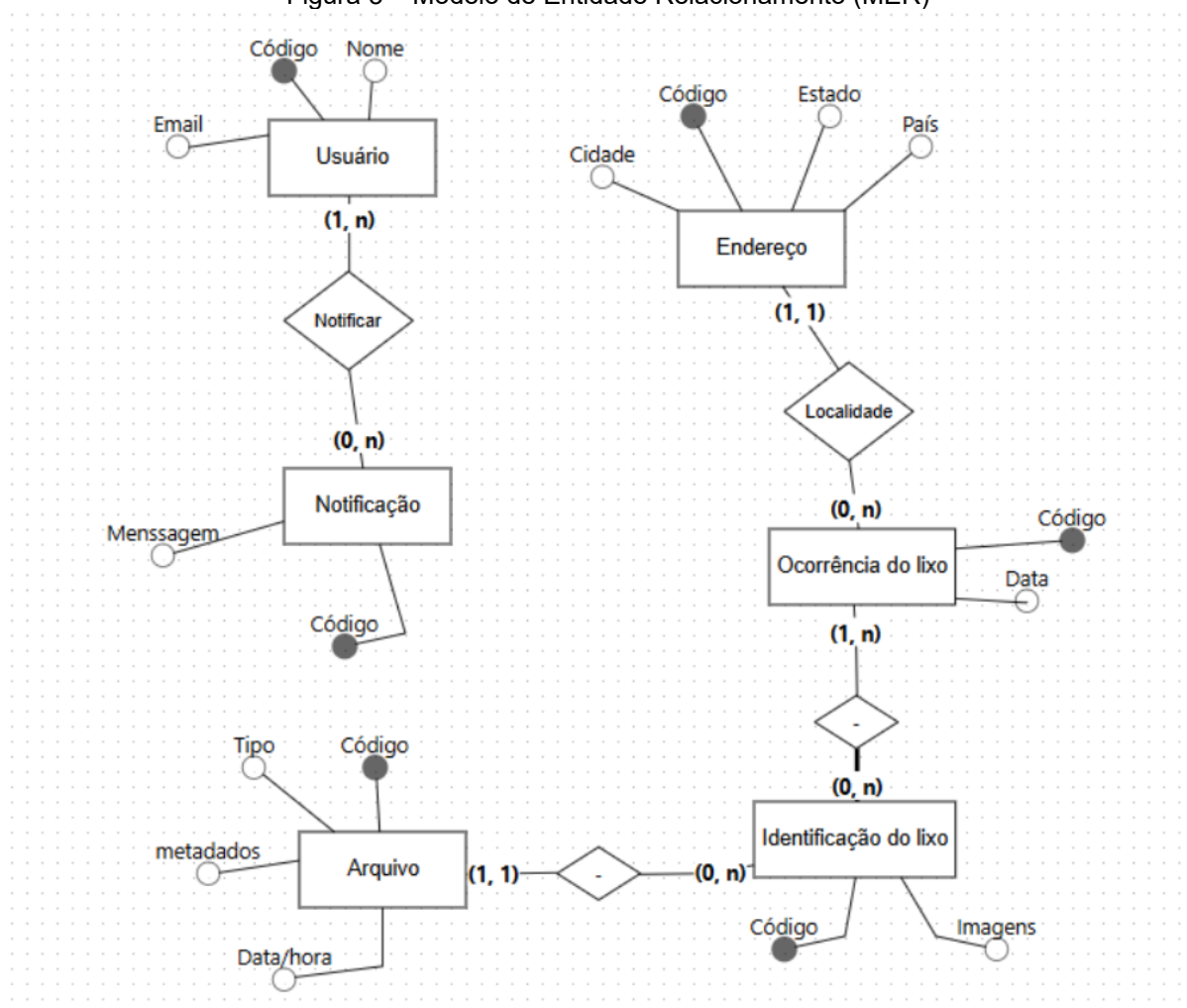


Fonte: Autoria própria, 2025

3.4 Projeto lógico do banco de dados

O Modelo de Entidade Relacionamento (MER) presente (figura 3) foi estruturado para refletir os principais elementos envolvidos no processo de identificação e registro de lixo, contemplando desde a captura das imagens até a notificação dos usuários.

Figura 3 – Modelo de Entidade Relacionamento (MER)



Fonte: Autoria própria, 2025

A seguir, uma breve descrição dos componentes principais:

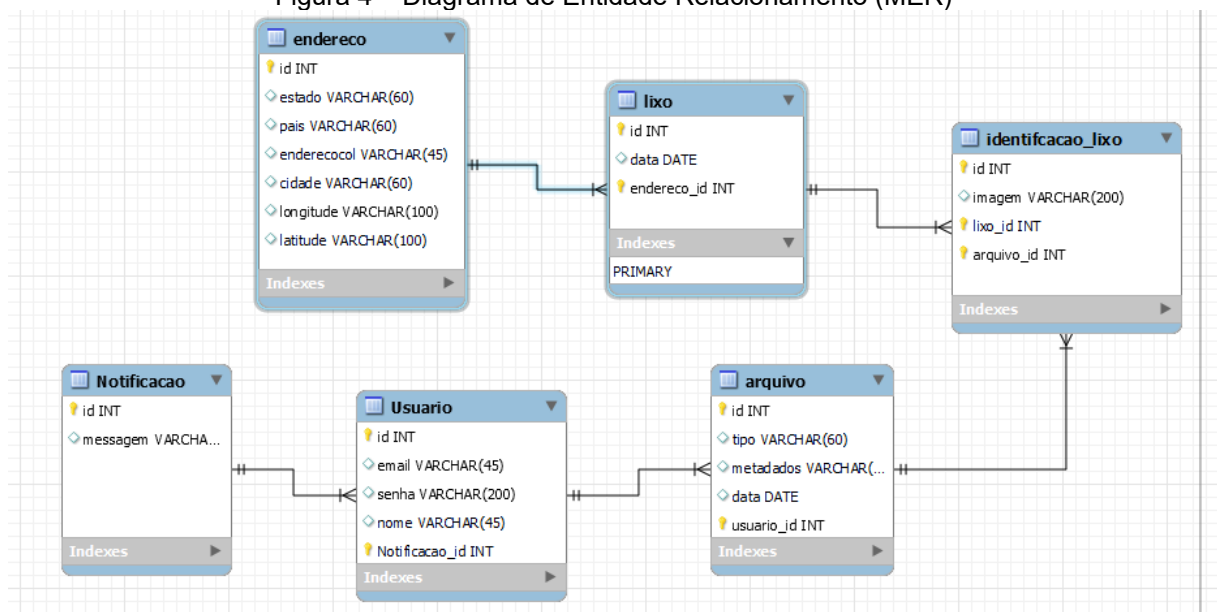
- **Usuário:** Representa os usuários que interagem com o sistema. Possui atributos como nome, e-mail.
- **Notificação:** Armazena informações sobre mensagens enviadas aos usuários, com data e conteúdo.
- **Endereço:** Detalham o local em que a ocorrência ocorreu, com atributos como cidade, estado e país.
- **Ocorrência do lixo:** Define quando ocorreu uma identificação de descarte irregular, possui a data de quando foi registrado.
- **Identificação do lixo:** Entidade que relaciona o arquivo, o lixo e a ocorrência do lixo. É associada a um ou mais arquivos (várias imagens).

- Arquivo: Registra metadados das evidências capturadas (fotos), incluindo tipo, data e outros detalhes técnicos.

3.5 Projeto físico do banco de dados

O Diagrama Entidade Relacionamento (DER) apresentado (figura 4) visa representar graficamente a estrutura lógica do banco de dados para o sistema de monitoramento de lixo por meio de arquivos de imagem e inteligência artificial.

Figura 4 – Diagrama de Entidade Relacionamento (MER)



Fonte: Autoria própria, 2025

A seguir, uma breve descrição dos componentes principais:

Usuário:

- Responsável por acessar o sistema.
- Atributos como nome, e-mail, senha, celular e imagem.
- Pode estar associado a notificações.

Endereço:

- Representa a localização do evento, com cidade, estado, país, longitude e latitude.
- Associado à entidade lixo.

Arquivo:

- Representa os arquivos (imagens) enviados pelo usuário.
- Contém metadados e caminho de armazenamento.
- Relaciona-se com a identificação do lixo e com o usuário que enviou.

Identificação do lixo:

- Representa a tentativa de identificar o lixo.
- Ela pode ou não conter uma identificação do lixo.
- Contém a imagem do lixo.

Lixo:

- Representa a ocorrência do lixo quando identificado.
- Contém a data e hora do local da ocorrência.
- Relaciona-se com endereço, de onde ocorreu.

Notificação:

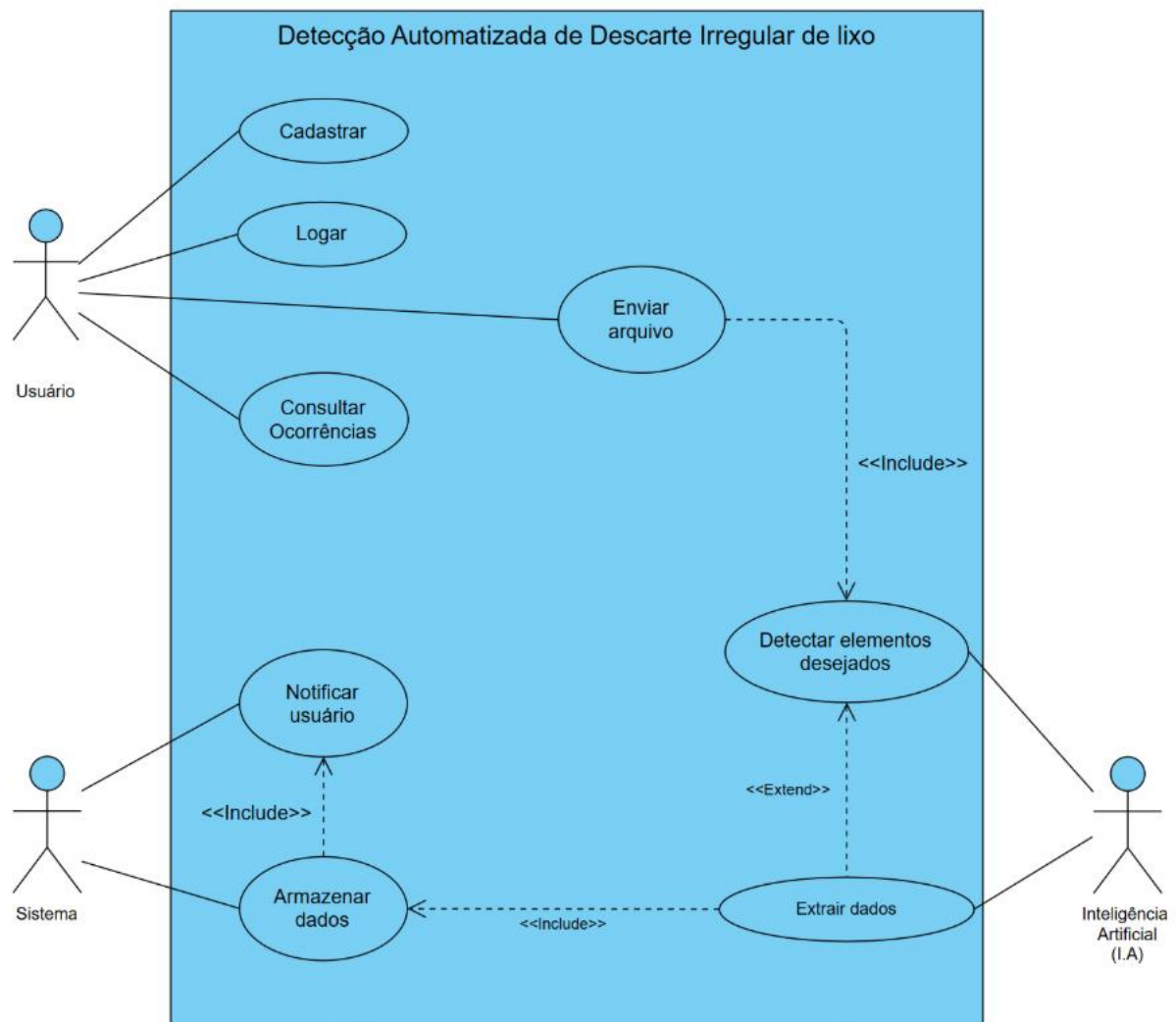
- Representa uma mensagem enviada pelo sistema ao usuário sobre ocorrência de uma identificação do lixo.

3.6 Modelagem do Software

A seção de modelagem do software apresenta os diagramas essenciais à representação funcional e estrutural do sistema. O diagrama de casos de uso expõe as funcionalidades disponibilizadas aos usuários e suas interações com o sistema. Em complemento, a documentação dos casos de uso formaliza os fluxos principais, as condições de execução e os tratamentos de exceções. O diagrama de classes modela as entidades e suas relações. Por fim, os diagramas de sequência ilustram os fluxos de mensagens entre os componentes do sistema em diferentes cenários operacionais.

3.6.1 Diagrama de Caso de Uso

Figura 5 – Diagrama de Caso de Uso



Fonte: Autoria própria, 2025

O diagrama de caso de uso, representa todos os requisitos funcionais do projeto, em ação com os devidos atores estabelecidos. Conforme o diagrama (Figura 5), o ator Usuário tem como requisitos funcionais, cadastrar, logar, consultar ocorrências no sistema, como também enviar arquivos de diferentes formatos de imagem (png, jpg, etc.).

Por sequência, o ator Inteligência Artificial, é obrigado a detectar o elemento desejado nas imagens, tais como diversos tipos de lixos descartados indevidamente em locais públicos. Com base nessa detecção, o sistema deve extrair as características relevantes do momento da ocorrência, utilizando-se da requisição para uma API que extraia o horário, data e endereço do ocorrido através dos metadados.

Por último, o ator Sistema, é encarregado de armazenar os dados previamente extraídos, de mesmo modo que notifica aos usuários do software sempre que uma nova detecção de lixo for detectada e registrada.

3.6.2 Documentação de caso de uso

Tabela 1 – Documentação de caso de uso 01 “Cadastrar”

Nome do caso de uso	UC01 – Cadastrar
Ator Principal	Usuário
Atores Secundários	
Resumo	Esse caso de uso descreve a etapa inicial em que o usuário realiza seu cadastro no sistema, por meio do preenchimento de suas credenciais
Pré-condições	O usuário não pode já estar logado para iniciar o cadastro
Pós-condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
1. Informar o nome	
	2. Verificar se o nome é válido
3. Informar o email	
	4. Verificar se o email é válido
5. Informar uma senha	
	6. Verificar se a senha é válida
	7. Armazenar os dados do usuário no banco de dados
Restrições/Validações	1. O nome deve conter apenas caracteres válidos e não pode estar vazio. 2. O email deve ter um formato válido e não estar cadastrado previamente. 3. A senha deve atender aos critérios mínimos de segurança (ex.: 6 caracteres ou mais).
Cenário de Exceção I – Email já cadastrado	
Ações do Ator	Ações do Sistema
	1. Informar que o email já está cadastrado
	2. Sugerir que o usuário faça login
Cenário de Exceção II – Nome, email e/ou senha Inválidos	
Ações do Ator	Ações do Sistema
	1. Comunicar ao usuário que existem campos inválidos
	2. Solicitar o preenchimento dos campos com dados válidos

Fonte: Autoria própria, 2025

Tabela 2 – Documentação de caso de uso 02 “Logar”

Nome do caso de uso	UC02 – Logar
Ator Principal	Usuário
Atores Secundários	
Resumo	Esse caso de uso descreve a etapa em que o usuário realiza seu login no sistema, através do preenchimento de suas credenciais.
Pré-condições	O usuário deve estar previamente cadastrado no sistema.
Pós-condições	O usuário será redirecionado à tela inicial após login bem-sucedido.
Cenário Principal	
Ações do Ator	Ações do Sistema
1. Informar o email	
	2. Verificar se o email é válido
3. Informar a senha	
	4. Buscar o email fornecido no banco de dados.
	5. Verificar se a senha fornecida está de acordo com a do banco de dados.
	6. Efetuar login do usuário.
Restrições/Validações	1. Para logar, é necessário ter feito um cadastro com o mesmo email.
Cenário de Exceção I – Email não encontrado no banco	
Ações do Ator	Ações do Sistema
	1. Comunicar ao usuário que o email fornecido não foi encontrado
	2. Solicitar o preenchimento dos campos com dados válidos
	3. Sugerir ao usuário para fazer o cadastro, caso não tenha feito
Cenário de Exceção II – Dados Inválidos	
Ações do Ator	Ações do Sistema
	1. Sistema comunica ao usuário que um dos seus dados está inválido
	2. Solicitar o preenchimento dos campos de texto com dados válidos

Fonte: Autoria própria, 2025

Tabela 3 – Documentação de caso de uso 03 “Enviar Arquivo”

Nome do caso de uso	UC03 – Enviar Arquivo
Ator Principal	Usuário
Atores Secundários	
Resumo	Esse caso de uso descreve as possíveis etapas que o usuário deve percorrer para efetuar o envio de um arquivo no sistema a fim de validar uma detecção de lixo.
Pré-condições	O usuário deve estar logado no sistema
Pós-condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
	1. Abrir o explorador de arquivos
2. Selecionar o arquivo	
	3. Verificar se o tipo de arquivo é válido
	4. Verificar se o tamanho do arquivo é válido
	5. Mandar o arquivo para a IA processar
	6. Exibir um ícone de carregamento ao usuário
Restrições/Validações	1. O arquivo deve estar no formato correto 2. O arquivo não deve ultrapassar o limite do tamanho
Cenário de Exceção I – Imagem inválida	
Ações do Ator	Ações do Sistema
	1. Informar ao usuário que a imagem selecionada é inválida
	2. Informar que a imagem deve estar em png, jpeg ou jpg
	3. Informar que a imagem não deve passar do tamanho de 5mb

Fonte: Autoria própria, 2025

Tabela 4 – Documentação de caso de uso 04 “Consultar Ocorrências de descarte irregular de lixo”

Nome do caso de uso	UC04 – Consultar Ocorrências de descarte irregular de lixo
Ator Principal	Usuário
Atores Secundários	
Resumo	Esse caso de uso descreve os passos necessários para o usuário realizar uma consulta das ocorrências registradas no banco de dados do sistema.

Pré-condições	O usuário deve estar logado no sistema
Pós-condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
1. Acessar a tela de mapa de consulta	
	2. Carregar a interface do mapa (Google Maps API)
	3. Consultar o banco de dados para obter todas as coordenadas de descartes registrados
	4. Plotar marcadores (pinos) no mapa correspondentes aos locais de ocorrências
	5. Exibir um modal com detalhes da ocorrência (foto do lixo, data, local e hora)
Restrições/Validações	1. Deve haver ao menos 1 ocorrência para ser consultada
Cenário de Exceção I – Login inválido / Sessão expirada	
Ações do Ator	Ações do Sistema
1. Tentar acessar a tela de consulta sem estar autenticado	
	2. Detectar ausência de credenciais ou token de sessão válido
	3. Exibir mensagem de erro comunicando que o usuário não está logado
	4. Solicitar login ou preenchimento de dados de acesso válido
Cenário de Exceção II – Nenhuma ocorrência encontrada	
Ações do Ator	Ações do Sistema
	1. Realizar a consulta no banco de dados e não obter retorno de registros
	2. Exibir mensagem informativa: "Nenhuma ocorrência foi encontrada"
	3. Manter o usuário na tela de consulta (mapa vazio ou limpo)

Fonte: Autoria própria, 2025

Tabela 5 – Documentação de caso de uso 05 “Detectar elementos desejados”

Tabela 5 – Documentação de caso de uso 05 – Detectar elementos desejados	
Nome do caso de uso	UC05 – Detectar elementos desejados
Ator Principal	Inteligência Artificial
Atores Secundários	Sistema
Resumo	Esse caso de uso descreve a função da Inteligência Artificial que o sistema deve detectar passivamente os elementos presentes em imagens, utilizadas como entrada.
Pré-condições	Receber arquivos de imagem como entrada
Pós-condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
1. Processar o(s) frame(s) necessários no modelo de visão computacional	
	2. Salvar e enviar os frames rotulados para próxima etapa
Restrições/Validações	1. Ter pelo menos 1 frame com elementos desejados no arquivo
Cenário de Exceção I – Nenhum elemento detectado	
Ações do Ator	Ações do Sistema
	1. Comunicar ao usuário que nenhum elemento desejado foi detectado no arquivo
	2. Solicitar envio de outro arquivo

Fonte: Autoria própria, 2025

Tabela 6 – Documentação de caso de uso 06 “Extrair dados”

Nome do caso de uso	UC06 – Extrair dados
Ator Principal	Inteligência Artificial
Atores Secundários	Sistema
Resumo	Esse caso de uso descreve a etapa em que a Inteligência Artificial realiza a extração de informações relevantes a partir dos frames previamente classificados como ocorrências.
Pré-condições	As ocorrências devem estar validadas pela IA
Cenário Principal	
Ações do Ator	Ações do Sistema
1. Recebe os frames classificados como ocorrência	
	2. Acessar os metadados do arquivo de imagem
	3. Extrair as tags de Latitude, Longitude e Data/Hora da captura
	4. Enviar as coordenadas para uma API de Geocodificação Reversa para obter o endereço
	5. Organizar os dados estruturados em formato JSON
	6. Enviar o pacote de dados para o módulo de armazenamento
Restrições/Validações	1. O arquivo de imagem deve conter metadados GPS íntegros. 2. A data da captura deve ser válida (não futura). 3. A API de geolocalização deve estar online para converter coordenadas em endereço.
Cenário de Exceção I – Ausência de Metadados GPS	
Ações do Ator	Ações do Sistema
1. Tentar ler as tags da imagem	
	2. Detectar ausência de coordenadas geográficas
	3. Marcar a localização como "Não identificada"

Fonte: Autoria própria, 2025

Tabela 7 – Documentação de caso de uso 07 “Armazenar dados”

Nome do caso de uso	UC07 – Armazenar dados
Ator Principal	Sistema
Resumo	Este caso de uso descreve o processo em que o sistema armazena, de forma estruturada e persistente, os dados do descarte detectado pela IA e os dados de localização extraídos. As informações salvas tornam-se imediatamente disponíveis para visualização no mapa do site.
Pré-condições	Dados de detecção e localização extraídos com sucesso.
Cenário Principal	
Ações do Ator	Ações do Sistema
1. Receber os dados estruturados da IA	
	2. Verificar se os dados estão completos e consistentes
	3. Estabelecer conexão segura com o banco de dados
	4. Inserir os dados em suas respectivas tabelas
	5. Confirmar a persistência dos dados no banco de dados
	6. Disponibilizar os dados para o <i>front-end</i> (mapa interativo) para consulta dos usuários
	7. Retornar confirmação de sucesso para disparar a notificação ao usuário
Restrições/Validações	1. Todos os campos obrigatórios devem estar preenchidos 2. O sistema deve impedir duplicação de registros idênticos para o mesmo evento.
Cenário Alternativo – Armazenamento parcial de dados	
Ações do Ator	Ações do Sistema
1. Receber dados estruturados da IA	
	2. Identificar falta de certos dados
	3. Rotular um ou mais campos como incompletos ou impossível de identificar.

Fonte: Autoria própria, 2025

Tabela 8 – Documentação de caso de uso 8 “Notificação”

Nome do caso de uso	UC8 – Notificação
Ator Principal	Sistema
Atores Secundários	
Resumo	Esse caso de uso descreve a etapa final do fluxo de denúncia, onde o sistema exibe ao usuário um feedback visual imediato (modal ou alerta) confirmando que o descarte irregular foi detectado e registrado com sucesso no mapa.
Pré-condições	Verificar se algum dado contendo alguma ocorrência foi guardada no banco de dados.
Pós-condições	
Cenário Principal	
Ações do Ator	Ações do Sistema
	1. Verificar se algum dado com ocorrência foi inserido no banco de dados.
	2. Resgatar os dados da ocorrência e as informações do usuário.
	3. Classificar os dados e configurar o tipo da mensagem.
	4. Enviar através de um middleware a notificação para o usuário de destino.
Restrições/Validações	1. Todas as informações devem estar preenchidas em suas tabelas correspondentes. 2. As informações devem conter uma semântica de valor coerente. 3. O endereço final da notificação deve ser um valor válido.
Cenário de Exceção I – Dados incompletos ou ilegíveis	
Ações do Ator	Ações do Sistema
	1. Verificar se existe algum valor como nulo ou como indefinido.
	2. Detectar se o middleware usado para o envio de mensagem obteve uma resposta de êxito ao enviar a mensagem.

Fonte: Autoria própria, 2025

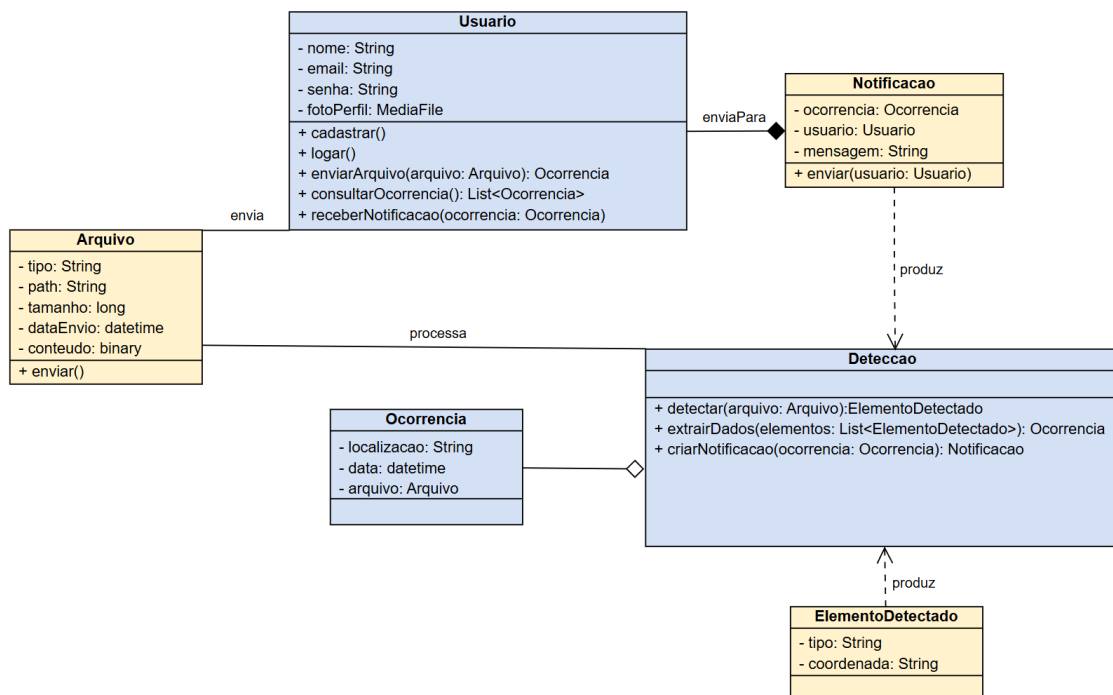
3.6.3 Diagrama de Classe

O Diagrama de Classes (Figura 5) é composto por classes que simulam a detecção automática de focos de descarte irregular de lixo. A classe Arquivo, que contém dados como tipo, caminho e conteúdo da mídia, é enviada por um Usuário, que pode se cadastrar, autenticar no sistema, enviar arquivos e consultar o histórico de Ocorrência.

A classe “Deteccao” é o núcleo do processo, responsável por analisar os arquivos de mídia. Ela detecta e delimita os “ElementoDetectado” (como resíduos e aglomerados de lixo) no conteúdo visual. A partir desses elementos, a classe extrai os dados contextuais para gerar uma Ocorrência, que é registrada com informações como localização, data e o arquivo de origem que a evidenciou.

Por fim, o sistema gera e despacha uma “Notificacao” relacionada à ocorrência registrada para o usuário responsável. O Diagrama de Classes reflete, portanto, um fluxo automatizado que integra desde a captura da imagem até a geração de um alerta, conectando o software de detecção com a interação do usuário final.

Figura 6 – Diagrama de Classes
Diagrama de classe
EcoDetect



Fonte: Autoria própria, 2025

3.6.4 Diagramas de Sequência

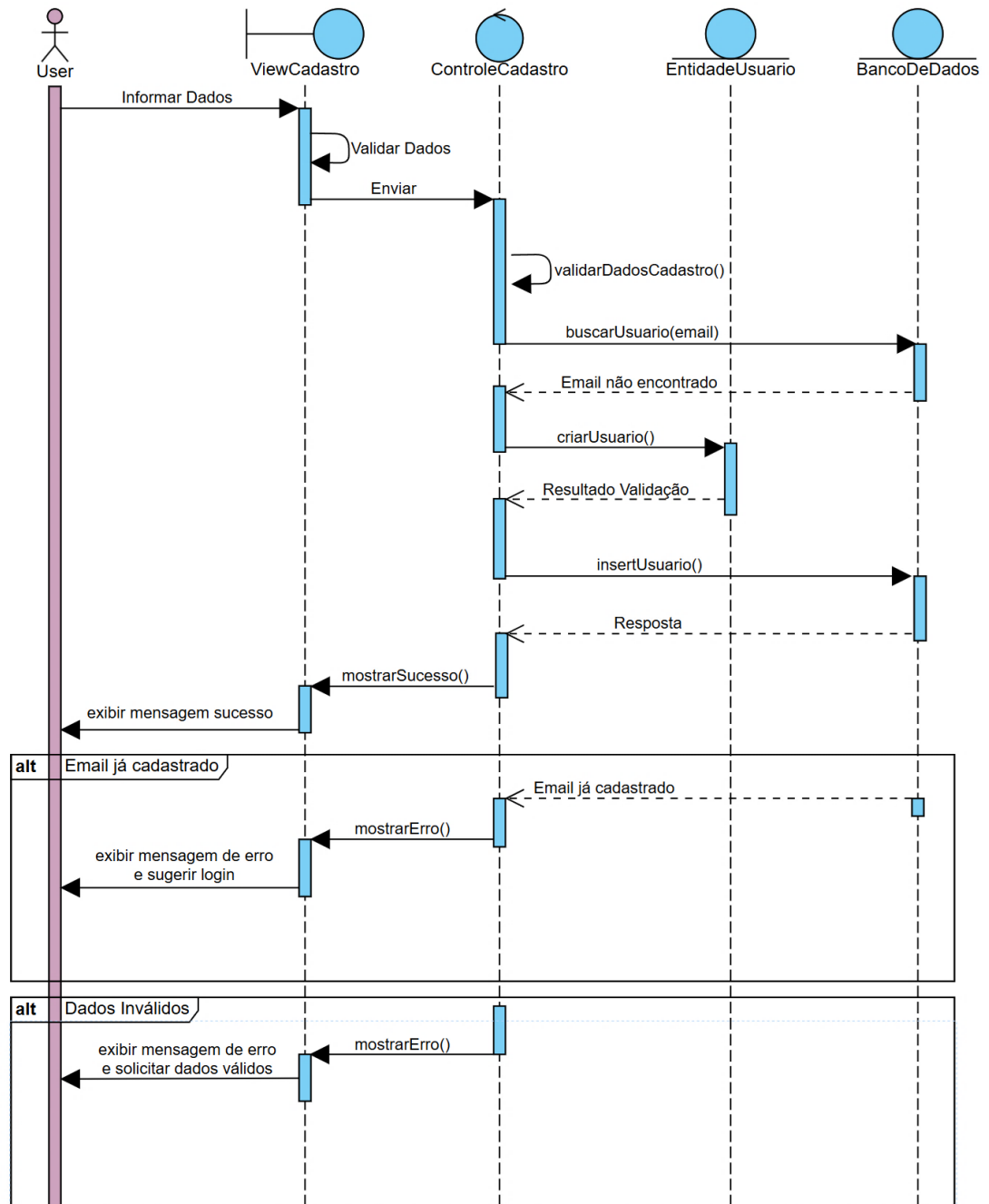
3.6.4.1 DS Cadastro

O diagrama de sequência "Cadastro" (figura 7) representa o fluxo em que o usuário fornece seus dados para realizar o registro no sistema. As informações são inseridas na interface e encaminhadas ao controlador, que realiza a validação dos dados. Estando os dados corretos, o sistema cria a instância do novo usuário e armazena suas informações no banco de dados. Ao final do processo, uma mensagem de confirmação é exibida ao usuário, indicando que o cadastro foi concluído com sucesso.

O diagrama também contempla dois cenários de exceção. O primeiro ocorre quando o e-mail informado já está cadastrado no sistema, impedindo a duplicidade e exigindo uma nova tentativa com outro endereço. O segundo cenário acontece quando algum dos dados fornecidos é inválido, levando o sistema a exibir uma mensagem de erro e solicitar que o usuário preencha novamente as informações de forma correta.

Figura 7 – Diagrama de Sequência “Cadastro”

sd cadastro



Fonte: Autoria própria, 2025

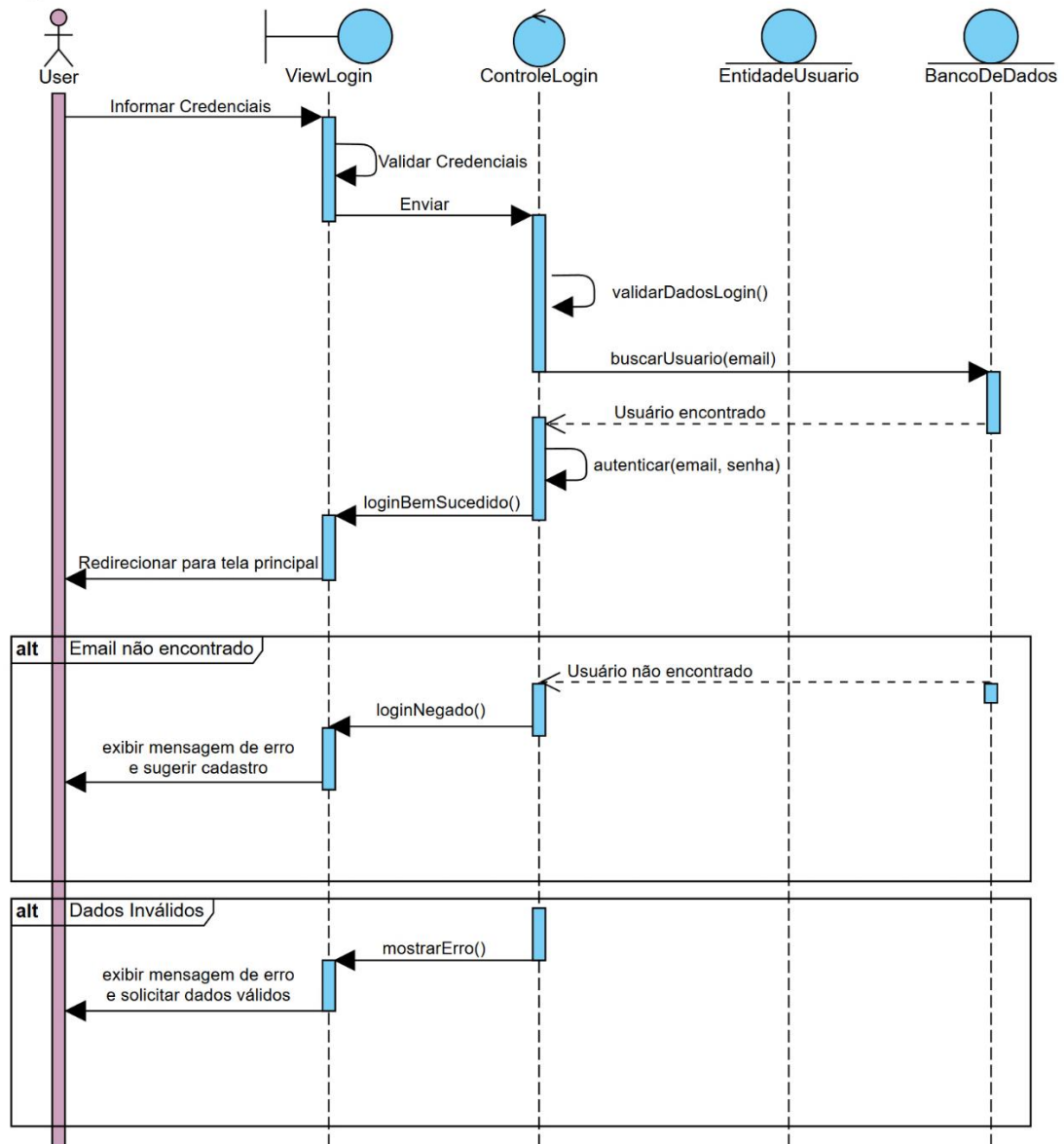
3.6.4.2 DS Login

O diagrama de sequência "Login" (figura 8) representa o fluxo em que o usuário informa suas credenciais para acessar o sistema. Após o envio das informações pela interface, os dados são validados e encaminhados ao controlador, que realiza a busca do usuário no banco de dados. Caso o usuário seja encontrado e as credenciais estejam corretas, a autenticação é realizada com sucesso, e o sistema redireciona o usuário para a tela principal.

O diagrama também contempla dois cenários de exceção. O primeiro ocorre quando o e-mail informado não está cadastrado no sistema, fazendo com que uma mensagem de erro seja exibida, acompanhada da sugestão de realizar o cadastro. O segundo cenário ocorre quando os dados preenchidos são inválidos ou inconsistentes, sendo então apresentada ao usuário uma mensagem de erro com a solicitação de correção.

Figura 8 – Diagrama de Sequência “Login”

sd login



Fonte: Autoria própria, 2025

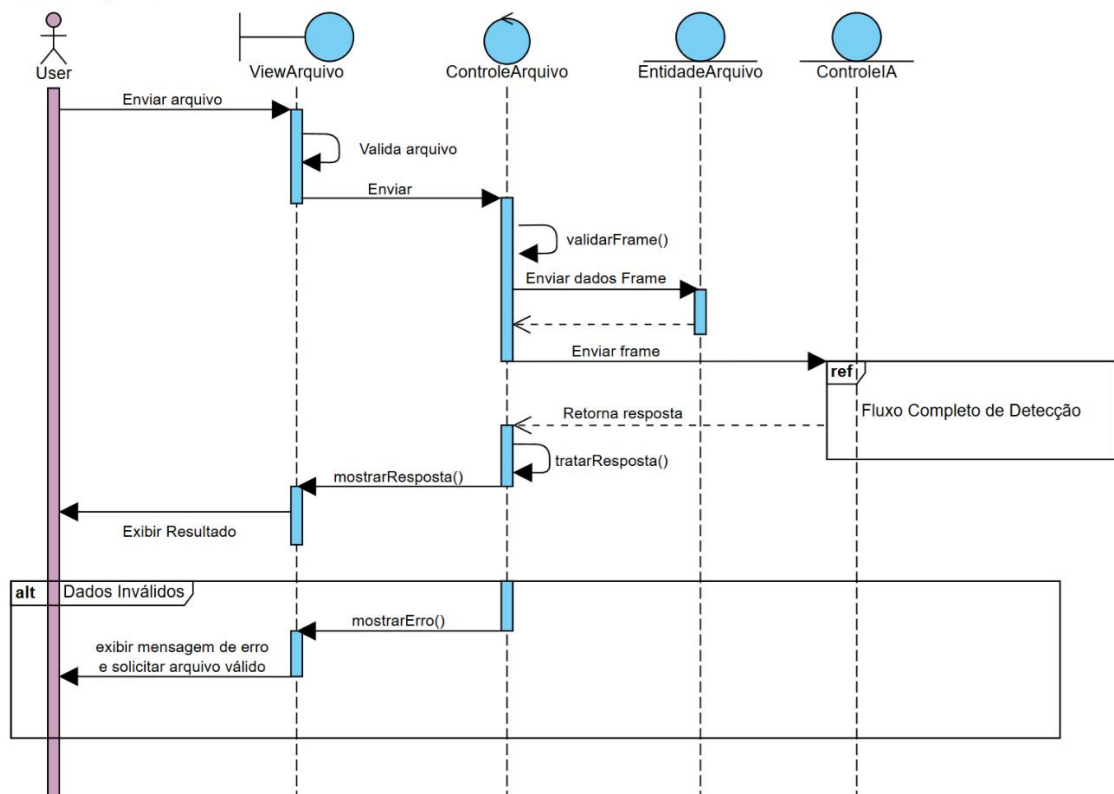
3.6.4.3 DS Enviar Arquivo

O diagrama de sequência "Enviar Arquivo" (figura 9) representa o processo em que o usuário realiza o envio de um arquivo para o sistema, contendo os dados visuais que serão analisados. O arquivo é inicialmente validado pela interface e, em seguida, encaminhado ao controlador responsável, que realiza a verificação do conteúdo e extrai os frames relevantes. Após essa etapa, os dados são direcionados ao módulo de detecção representado por um diagrama de sequência separado, identificado no modelo pelo fragmento de referência "ref".

Ao final do processamento, uma resposta é gerada e retornada ao controlador, que trata o resultado e envia o retorno para o usuário. O fluxo também contém um cenário de exceção, no qual o arquivo enviado contém dados inválidos. Nessa situação, o sistema informa o erro e solicita ao usuário o reenvio de um arquivo válido.

Figura 9 – Diagrama de Sequência "Enviar Arquivo"

sd Enviar Arquivo



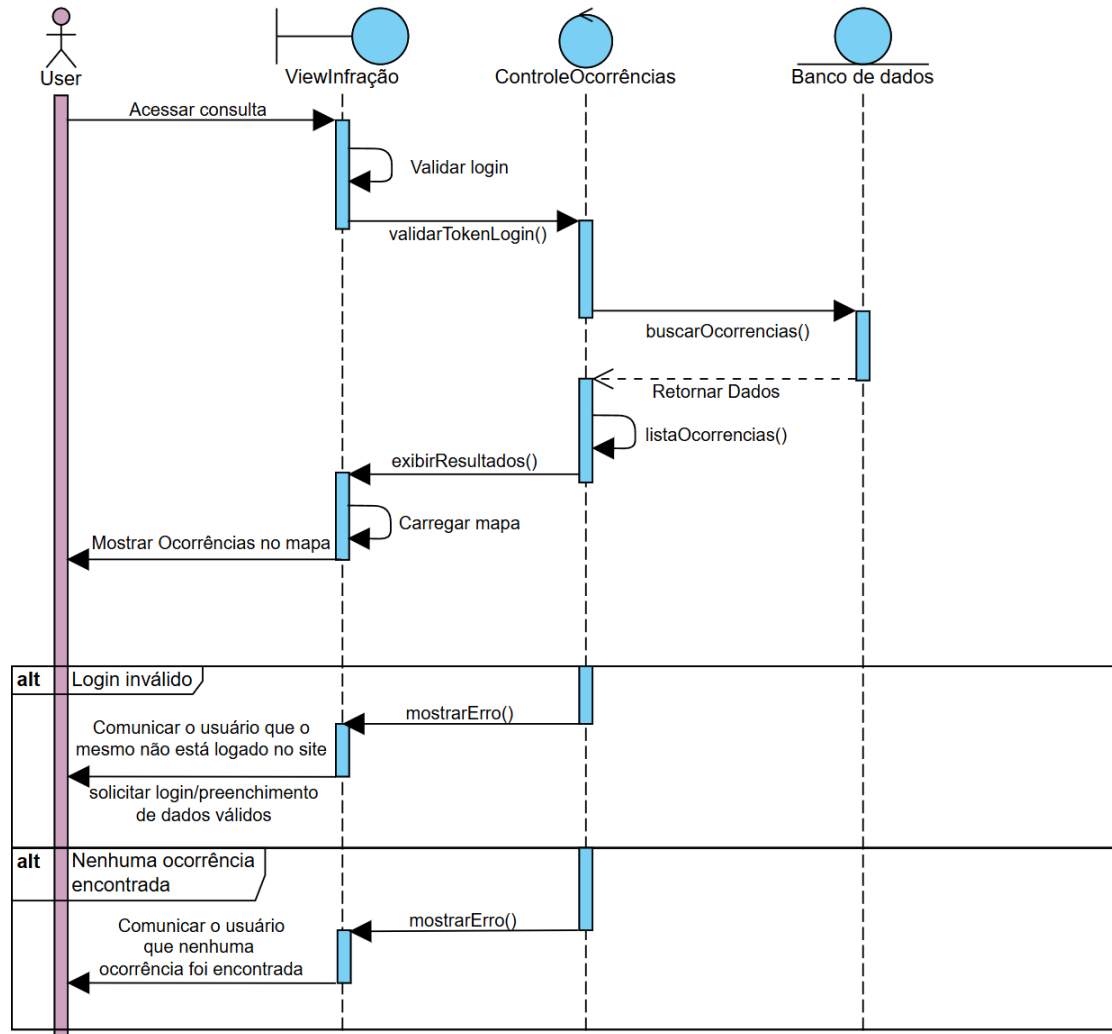
Fonte: Autoria própria, 2025

3.6.4.4 DS Consultar Ocorrências

O diagrama de sequência "Consultar Ocorrências" (Figura 10) representa o fluxo de interação em que o usuário, após acessar a funcionalidade de consulta, visualiza a interface "ViewOcorrência". O sistema então valida automaticamente se o usuário possui uma sessão ativa "validarTokenLogin()". Caso o login seja válido, o sistema prossegue para buscar os registros no banco de dados "buscarOcorrencias()". Se existirem ocorrências associadas ao usuário, os dados são retornados "listaOcorrencias()" e exibidos em um mapa, onde são plotadas geograficamente para visualização "Carregar mapa e exibirResultados()".

O fluxo também contempla dois cenários de exceção. O primeiro ocorre quando o token de login é inválido ou o usuário não está autenticado, o que aciona uma mensagem de erro solicitando que ele faça login ou preencha credenciais válidas. O segundo cenário trata da ausência de ocorrências registradas para um usuário válido, onde o sistema comunica que nenhuma ocorrência foi encontrada.

Figura 10 – Diagrama de Sequência “Consultar Ocorrências”
sd Consultar Ocorrências



Fonte: Autoria própria, 2025

3.6.4.5 DS Fluxo completo de detecção

O diagrama de sequência "Fluxo completo de detecção" (Figura 11) representa o núcleo funcional do sistema de monitoramento automatizado de descarte irregular de lixo, sendo o principal responsável pela identificação e registro de ocorrências a partir de imagens capturadas. Este fluxo ocorre de forma contínua, iniciando com um laço de repetição (loop) que processa cada imagem para o monitoramento.

O processo tem início com a chamada do caso de uso de detecção de elementos "detectarElementos() - UC05", no qual o módulo de IA analisa o frame com o objetivo de identificar a presença de resíduos ou aglomerados de lixo.

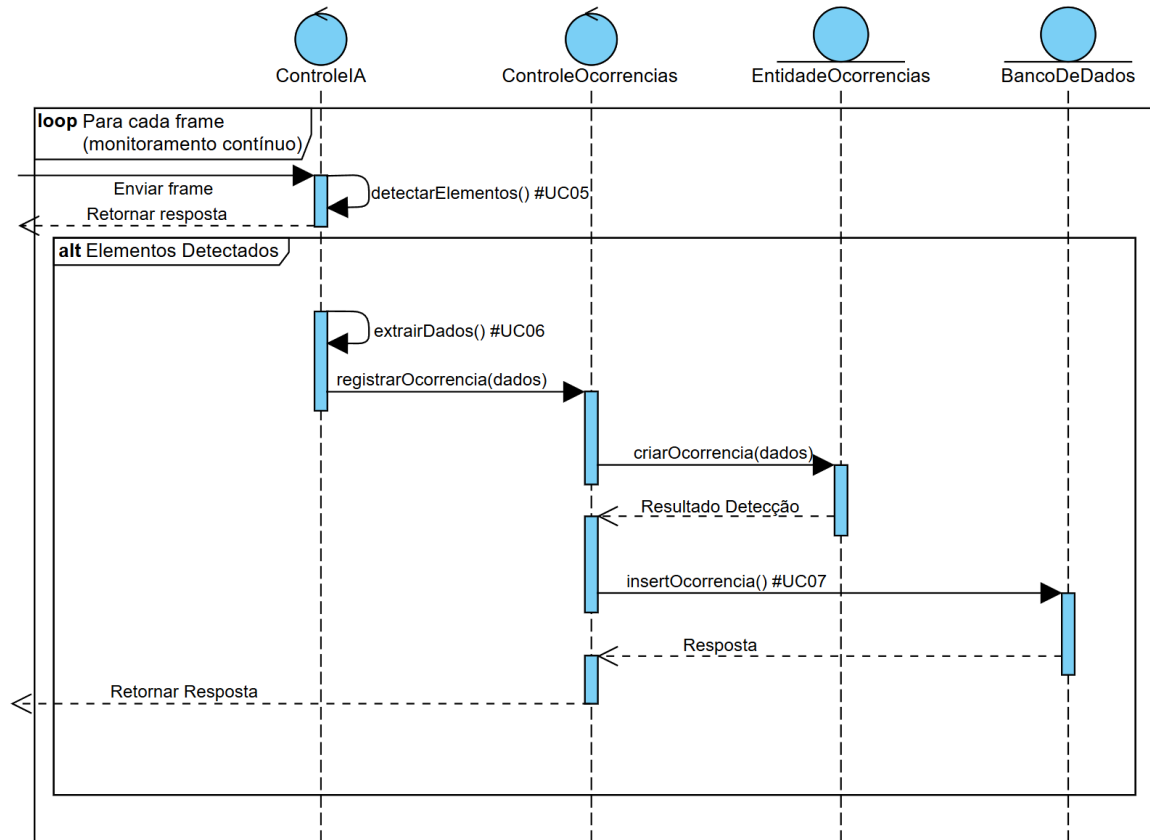
Existem então dois caminhos possíveis:

-Cenário Alternativo: Nenhum elemento detectado - Caso nenhum resíduo seja identificado, o fluxo é interrompido para aquele frame, retornando uma resposta negativa para o sistema.

-Cenário Principal: Elementos Detectados - Se a detecção for bem-sucedida, o fluxo avança para a próxima etapa. Nesta fase, os dados contextuais dos elementos detectados são consolidados "extrairDados() - UC06" e, em seguida, uma ocorrência é formalmente criada e registrada no banco de dados "registrarOcorrencia() - UC07".

Uma vez registrada, a ocorrência fica persistida para consultas e ações futuras, e uma resposta conclusiva é retornada, sinalizando o sucesso da operação. Este modelo dinâmico e automatizado garante que o sistema opere de forma contínua e eficiente, identificando e registrando focos de descarte irregular sem a necessidade de intervenção humana direta.

Figura 11 – Diagrama de Sequência “Fluxo completo de detecção”
sd Fluxo completo de detecção



Fonte: Autoria própria, 2025

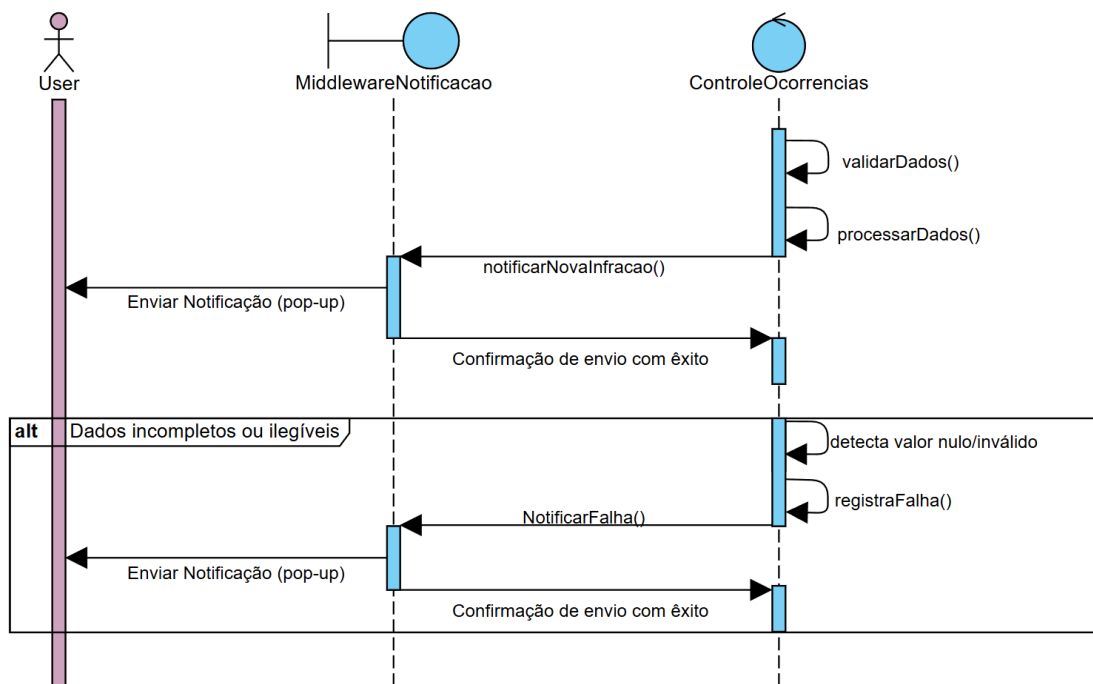
3.6.4.6 DS Notificação

O diagrama de sequência "Notificação" (figura 12) descreve o fluxo responsável por enviar ao usuário uma notificação relacionada ao registro de uma nova Ocorrência. A partir dos dados processados pelo controlador de Ocorrências, uma solicitação é enviada ao middleware responsável pela entrega, que então dispara o envio da notificação ao usuário por pop-up, confirmando o sucesso da operação.

O diagrama também contempla um cenário de exceção. Caso os dados recebidos estejam incompletos ou ilegíveis, o sistema detecta a falha, registra o erro e executa uma notificação alternativa informando a ocorrência do problema. Ainda assim, há uma confirmação de envio, assegurando que o usuário será informado da tentativa, mesmo que sem sucesso total.

Figura 12 – Diagrama de Sequência “Notificação”

sd Notificação



Fonte: Autoria própria, 2025

4 PROJETO DO PROGRAMA

A arquitetura do sistema foi projetada seguindo o modelo Cliente-Servidor (*Client-Server*). Esta abordagem separa as responsabilidades da aplicação em duas partes distintas: o Front-End (Cliente), que é a interface com a qual o usuário interage (o site), e o Back-End (Servidor), que centraliza o processamento de dados, a lógica de negócio e a inteligência artificial.

4.1 Back-End (Servidor)

No back-end, que atua como o "cérebro" da aplicação, a linguagem de programação escolhida foi o Python. Esta decisão foi motivada pelo seu robusto ecossistema de bibliotecas para Inteligência Artificial (IA) e Visão Computacional, essenciais para o núcleo do projeto.

Para a construção da API (Interface de Programação de Aplicações), foi adotado o framework FastAPI, conhecido por sua alta performance e facilidade na estruturação de rotas. As principais rotas (*endpoints*) desenvolvidas incluem:

- Gerenciamento de Usuários: Rotas para a criação de novas contas e para a autenticação (login) dos usuários.
- Detecção de Resíduos: A rota principal do projeto, responsável por receber uma imagem enviada pelo cliente. (A descrição detalhada do processamento contida na próxima seção, 4.1.1.)

4.1.1 Fluxo de processamento da imagem

Quando a rota de Detecção de Resíduos é acionada, ela executa a análise e processamento da imagem em três etapas principais:

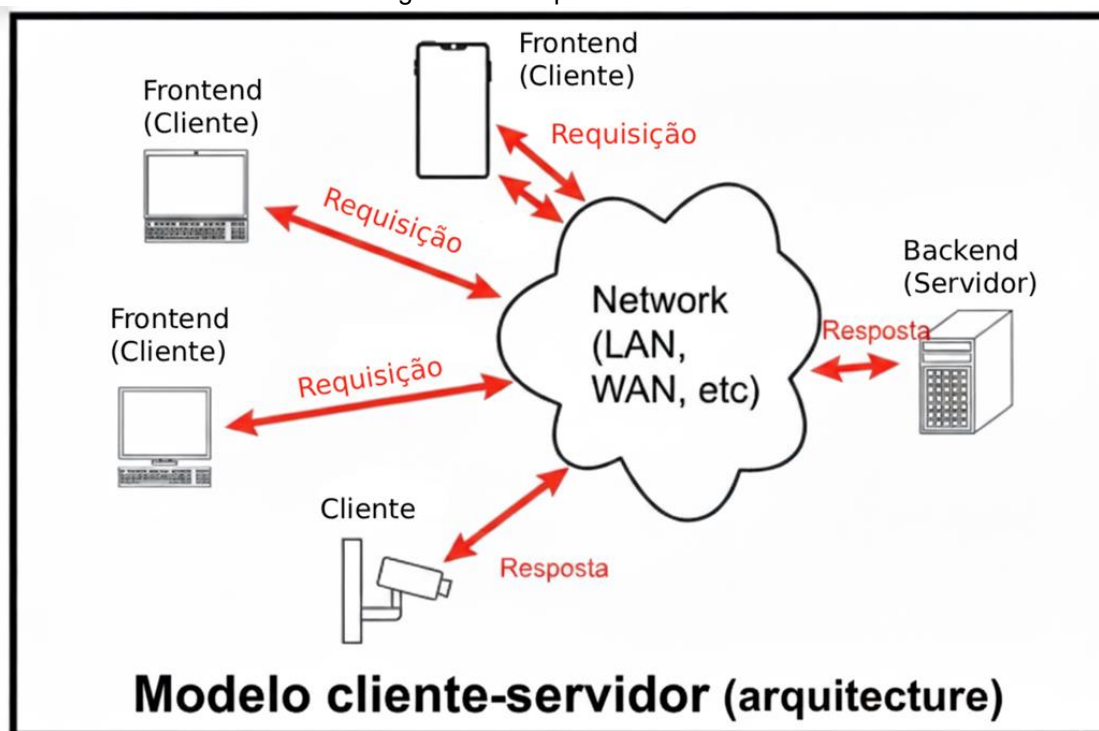
- Detecção dos Elementos (YOLO): O núcleo da análise é um modelo de detecção de objetos YOLO (You Only Look Once), treinado com um conjunto de dados específico (via Roboflow) para identificar focos de descarte irregular. A obtenção da imagem processada (com ou sem elementos detectados) é feita por meio de uma requisição (*Inference API*) ao modelo hospedado no Roboflow.

- **Extração e Conversão de Dados:** Em paralelo ou subsequente à detecção, são utilizadas bibliotecas especializadas para a extração e validação dos dados de contexto. A biblioteca PIL.ExifTags é empregada para extrair os metadados EXIF, incluindo as coordenadas de GPS contidas na imagem.
- **Geolocalização:** O sistema está integrado a uma API externa (via *geopy.geocoders*, utilizando o *Nominatim*) que processa e valida as coordenadas obtidas, convertendo-as em um endereço físico legível para o mapeamento e registro do descarte.

Para a persistência dos dados — como informações de usuários, históricos de envios e as localizações geográficas dos descartes confirmados — foi utilizado o sistema de gerenciamento de banco de dados relacional MySQL.

Conforme a arquitetura apresentada (Figura 5) Cliente-Servidor, o back-end atua como o Servidor. Ele não possui interface visual; sua função é aguardar requisições do cliente. Ele centraliza a lógica de negócio (autenticação), a execução do modelo de IA (análise da imagem) e a comunicação direta com o banco de dados (MySQL). Ao receber uma foto, ele a processa e retorna ao cliente uma resposta estruturada (ex: "Lixo detectado" ou "Nenhum lixo encontrado"). (Figura 13)

Figura 13 – Arquitetura Cliente-Servidor



4.2 Front-End (Cliente)

O front-end, a interface web com a qual o usuário interage, foi desenvolvido utilizando as tecnologias fundamentais da web: HTML para a estruturação do conteúdo, CSS para a estilização visual e o JavaScript para a interatividade da página.

Através desta interface, o usuário pode se cadastrar, fazer login e, o mais importante, enviar uma imagem (capturada pela câmera ou da galeria) para análise do sistema.

Uma funcionalidade crucial do front-end é a exibição dos dados de localização. Para isso, foi integrada a API do Google Maps. Esta API permite que a aplicação plote visualmente em um mapa interativo todos os pontos de descarte irregular que foram identificados e registrados pelo back-end, utilizando as coordenadas armazenadas no banco de dados.

Relação Cliente-Servidor (Front-End): A aplicação web funciona como o Cliente. O navegador do usuário é responsável por carregar os arquivos (HTML, CSS, JS) e renderizar a interface. O JavaScript no lado do cliente gerencia as ações do usuário; ele formula uma requisição HTTP para o servidor (back-end) contendo a imagem a ser analisada. Após o servidor processar e responder, o JavaScript no cliente recebe essa resposta e atualiza a interface para o usuário (ex: exibindo uma mensagem de sucesso e atualizando o mapa com o novo ponto).

5 CÓDIGO FONTE

Devido ao grande tamanho do código (~43Mb), a alta quantidade de arquivos presentes e o limite de 10Mb estabelecidos pelo site da unip, foi resolvido, para facilitar a visualização do código, anexar aqui os links para os repositórios GitHub onde o código está salvo.

Como também, um link para o download dos mesmos hospedados no Google Drive no email de um dos integrantes:

<https://drive.google.com/drive/folders/1HPes0htlNRCjmw1sR22NoJtEPVxCSpZB?usp=sharing>

No perfil do integrante Guilherme Gomes, está contido o repositório com todos os arquivos relacionados ao Front-end de nosso aplicativo. Segue o link:

<https://github.com/guilhermeAbbenante/APS-EcoDetect-FrontEnd>

Enquanto no perfil do integrante Hitalo Chaves, está contido o repositório com todos os arquivos relacionados ao Back-end de nosso aplicativo. Segue o link:

<https://github.com/Hitalo-27/back-deteccao-lixo>

6 REFERÊNCIA BIBLIOGRAFICA

ALPAYDIN, Ethem. **Introduction to machine learning**. MIT press, 2020.

BERTOLETI, Pedro. **Contagem de objetos em movimento com OpenCV e Python usando Raspberry Pi**. Embarcados, 21 de agosto de 2017. Disponível em: <https://embarcados.com.br/objetos-opencv-e-python-raspberry-pi/>. Acesso em: 24 de abril de 2025.

BIANCHI, Reinaldo Augusto da Costa. **Uso de heurísticas para a aceleração do aprendizado por reforço**. Tese de Doutorado. Universidade de São Paulo, São Paulo, 2004. Disponível em: <https://fei.edu.br/~rbianchi/publications/tese-bianchi.pdf>. Acesso em: 22 de abril de 2025.

BISHOP, Christopher M.; NASRABADI, Nasser M. **Pattern recognition and machine learning**. New York: springer, 2006.

GOMES, Dennis dos Santos. **Inteligência Artificial: conceitos e aplicações**. Revista Olhar Científico, v. 1, n. 2, p. 234-246, 2010.

GOODFELLOW, Ian et al. **Deep learning**. Cambridge: MIT press, 2016.

HASTIE, Trevor et al. **The elements of statistical learning**. 2009.

HAYKIN, Simon. **Redes neurais: princípios e prática**. Bookman Editora, 2001.

IBM. **What is computer vision?**. 2019. Disponível em: <https://www.ibm.com/think/topics/computer-vision>. Acesso em: 23 de abril de 2025.

KUNDU, Rohit. **YOLO: Algorithm for Object Detection Explained [+Examples]. What is YOLO architecture and how does it work? Learn about different YOLO algorithm versions and start training your own YOLO object detection models**.

V7Labs. 17 de janeiro de 2023. Disponível em: <https://www.v7labs.com/blog/yolo-object-detection>. Acesso em: 24 de abril de 2025.

MITCHELL, Tom M.; MITCHELL, Tom M. **Machine learning**. New York: McGraw-hill, 1997.

REDMON, Joseph et al. **You only look once: Unified, real-time object detection**. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2016. p. 779-788. Disponível em: https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Redmon_You_Only_Look_CV_PR_2016_paper.pdf. Acesso em: 23 de abril de 2025.

ROBOFLOW, **Explore the Roboflow Universe**. 2025. Disponível em: <https://universe.roboflow.com>. Acesso em 18 de abril de 2025

RUSSELL, Stuart J.; NORVIG, Peter. **Artificial intelligence: a modern approach**. pearson, 2016.

SRIVASTA, Niharika. **What is SOTA in Artificial Intelligence?**. E2E Networks. 2 de abril de 2025. Disponível em: <https://www.e2enetworks.com/blog/what-is-sota-in-artificial-intelligence>. Acesso em: 22 de abril de 2025.

TAULLI, Tom. **Introdução à inteligência artificial: uma abordagem não técnica**. Novatec Editora, 2020.

ULTRALYTICS. **Home**. 2025. Disponível em: <https://docs.ultralytics.com/>. Acesso em: 21 de abril de 2025

ULTRALYTICS. **Ultralytics YOLO11**. Disponível em: <https://docs.ultralytics.com/pt/models/yolo11/>. Acesso em: 21 de abril de 2025

VINA, Abirami. **Comparing Ultralytics YOLO11 vs previous YOLO models.**

Ultralytics. 2 de abril de 2025. Disponível em:

<https://www.ultralytics.com/blog/comparing-ultralytics-yolo11-vs-previous-yolo-models>. Acesso em: 22 de abril de 2025

FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

Atividades Práticas Supervisionadas (laboratórios, atividades em biblioteca, Iniciação Científica, trabalhos Individuais e em grupo, práticas de ensino e outras)

NOME: Guilherme Vieira Abbenante Gomes

RA: N8959E0 **CURSO:** Ciências da Computação CC8P33

CAMPUS: UNIP - TATUAPÉ

SEMESTRE: 8º Semestre

TURNO: NOTURNO

[illegible]

TOTAL DE HORAS: 80 HORAS

FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

Atividades Práticas Supervisionadas (laboratórios, atividades em biblioteca, Iniciação Científica, trabalhos Individuais e em grupo, práticas de ensino e outras)

NOME: Hitalo Chaves dos Santos

RA: N841419 CURSO: Ciências da Computação CC8P33

CAMPUS: UNIP - TATUAPÉ

SEMESTRE: 8º Semestre

__TURNO: NOTURNO

[illegible]

TOTAL DE HORAS: 80 HORAS

FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

Atividades Práticas Supervisionadas (laboratórios, atividades em biblioteca, Iniciação Científica, trabalhos Individuais e em grupo, práticas de ensino e outras)

NOME: João Victor Crisci

RA: F3445H0 CURSO: Ciências da Computação CC8P33

CAMPUS: UNIP - TATUAPÉ

SEMESTRE: 8º Semestre

TURNOS: NOTURNO

[illegible]

TOTAL DE HORAS: 80 HORAS